Ministère de l'Enseignement Supérieur et de la Recherche SCIENTIFIQUE
DIRECTION DES INSTITUTS SUPERIEURS DES ETUDES TECHNOLOGIQUE
ISET KAIROUAN

Les Fondamentaux de Kubernetes

Préparation à la certification CKA

Elaboré par :

Noureddine GRASSA

Année universitaire : 2023-2024

Tables des matières :

Chapitre 1 5

Introduction..... 5

1 Laboratoires..... 5

Exercice 1.1 : Configuration du système pour sudo 5

Chapitre 2 7

Les bases de Kubernetes 7

2 Laboratoires..... 7

Exercice 2.1 : Afficher les ressources en ligne 7

Chapitre 3 9

Installation et configuration 9

3 Laboratoires..... 9

Exercice 3.1 : Installer Kubernetes 9

Exercice 3.2 : Éléments de post installation 13

Exercice 3.3 : Déployer une application simple 14

Exercice 3.4 : Accès depuis l'extérieur du cluster 20

Chapitre 4 23

Architecture de Kubernetes 23

4 Laboratoires.....23

Conception : Les nœuds de base dans une architecture Kubernetes 23

Exercice 4.1 : Sauvegarder la base de données etcd 23

Chapitre 5 33

Gestion d'état avec les déploiements 33

5 Laboratoires.....33

Exercice 5.1 : Travailler avec des ReplicaSets.....33

Exercice 5.2 : Utilisation des déploiements 38

Exercice 5.3 : Mises à jour et restaurations à l'aide du déploiement 40

Exercice 5.4 : Travailler avec les DaemonSets.....42

Exercice 5.5 : Mises à jour et restaurations progressives à l'aide de DaemonSet 44

Chapitre 6 49

Volumes et données.....49

6 Laboratoires 49

Exercice 6.1 : Créer une ConfigMap 49

Exercice 6.2 : Créer un Persistant Volume (PV) 53
Exercice 6.3 : Créer un Persistant Volume Claim (PVC) 55
Exercice 6.3 : Créer un Pod utilisant un PVC 56
Chapitre 7 57
Services 57
7 Laboratoires 57
Exercice 7.1 : Qu'est-ce qu'un service dans Kubernetes 57
Exercice 7.2 : Configurer un service ClusterIP 59
Exercice 7.3: Configurer un service NodePort 63
Exercice 7.4: Configurer un service LoadBalancer sur EKS de AWS 66
Chapitre 8 72
Scheduling (Planification) 72
8 Laboratoires 72
Exercice 8.1: Attribuer des pods à l'aide des labels (étiquettes) 72
Exercice 8.2 : Utiliser les Taints pour contrôler le dépoloiement des pods 78
Chapitre 9 83
Journalisation et Débogage (Logging and Troubleshooting) 83
9 Laboratoires 83
Exercice 9.1 : Les emplacements des fichiers journaux 83
Exercice 9.2 : Afficher des résultats des journaux 86
Exercice 9.3 : Ajout d'outils de surveillance et de métriques 87
Chapitre 10 92
Sécurité 92
10 Laboratoires 92
Exercice 10.1 : Authentification et autorisation 92
Exercice 10.2 : Travailler avec TLS 99
ANNEXE 103
Exercice A.1 : CKA Simulator Kubernetes 1.29 104
Webographie 105

Liste des Figures :

Figure I : Script pour monter un cluster Kubernetes *11*
Figure II : Travailler online sur Killercoda *13*
Figure III : Utilisation de ngrok pour *21*
Figure IV : Accès externe via navigateur *22*
Figure V : Architecture des clusters Kubernetes *23*
Figure VI : Service ClusterIP *57*
Figure VII : Service NodePort *58*
Figure VIII : Service LoadBalancer *59*
Figure IX : Creation cluster Kubernetes sur AWS *66*
Figure X : Cluster Kubernetes avec un seul nœud *67*
Figure XI : Visualisation des pods sur le cluster Kubernetes EKS de AWS *68*
Figure XII: Visualisation des services dans EKS de AWS *69*
Figure XIII: L'URL pour accéder à la page web dans un navigateur *70*
Figure XIV: Affichage de la page NGINX avec L'URL fourni dans un navigateur *71*
Figure XV: L'architecture du pipeline des métriques de ressources *87*

Chapitre 1

Introduction

1 Laboratoires

Exercice 1.1 : Configuration du système pour sudo

Il est très dangereux d'exécuter un shell root sauf en cas d'absolue nécessité : une simple faute de frappe ou autre erreur peut causer des dommages graves et irréversibles.

Ainsi, la procédure judicieuse consiste à configurer les choses de telle sorte que des commandes uniques puissent être exécutées avec les privilèges du super utilisateur, en utilisant le mécanisme sudo . Avec sudo, l'utilisateur n'a besoin que de connaître son propre mot de passe et n'a jamais besoin de connaître le mot de passe root.

Si vous utilisez une distribution telle qu'Ubuntu, vous n'aurez peut-être pas besoin de suivre cet atelier pour configurer correctement sudo pour le cours.

Cependant, vous devez toujours vous assurer de bien comprendre la procédure.

Pour vérifier si votre système est déjà configuré pour permettre au compte utilisateur que vous utilisez d'exécuter sudo, exécutez simplement une commande simple comme :

```
$ sudo ls
```

Vous devriez être invité à saisir votre mot de passe utilisateur, puis la commande doit s'exécuter. Si au contraire vous obtenez un message d'erreur, vous devez exécuter la procédure suivante.

Lancez un shell root en tapant su puis en donnant le mot de passe root , pas votre mot de passe utilisateur.

Sur toutes les distributions Linux récentes , vous devez accéder au sous-répertoire /etc/sudoers.d et créer un fichier, généralement avec le nom de l'utilisateur à qui root souhaite accorder l'accès sudo . Cependant, cette convention n'est pas réellement nécessaire car sudo analysera tous les fichiers de ce répertoire selon les besoins. Le fichier peut simplement contenir :

```
étudiant ALL=(ALL) ALL
```

si l'utilisateur est étudiant.

Une pratique plus ancienne (qui fonctionne certainement toujours) consiste à ajouter une telle ligne à la fin du fichier /etc/sudoers. Il est préférable de le faire en utilisant le programme visudo , qui veille à ce que vous utilisiez la bonne syntaxe dans votre édition.

Vous devrez probablement également définir les autorisations appropriées sur le fichier en tapant :

```
$ sudo chmod 440 /etc/sudoers.d/student
```

(Notez que certaines distributions Linux peuvent nécessiter 400 au lieu de 440 pour les autorisations.)

Après avoir effectué ces étapes, quittez le shell racine en tapant exit , puis réessayez sudo ls .

Il existe de nombreuses autres façons pour un administrateur de configurer sudo, notamment en spécifiant uniquement certaines autorisations pour certains utilisateurs, en limitant les chemins recherchés, etc. Le fichier /etc/sudoers est très bien documenté.

Cependant, il existe un paramètre supplémentaire que nous vous recommandons fortement d'effectuer, même si sudo est déjà configuré sur votre système. La plupart des distributions établissent un chemin différent pour rechercher les exécutables pour les utilisateurs normaux et pour les utilisateurs root. En particulier, les répertoires /sbin et /usr/sbin ne sont pas recherchés, puisque sudo hérite du PATH de l'utilisateur, et non de l'utilisateur root complet.

Ainsi, dans ce cours, nous devrons constamment vous rappeler le chemin d'accès complet à de nombreux utilitaires d'administration système ; toute amélioration de la sécurité ne vaut probablement pas la peine de taper davantage et de déterminer dans quels répertoires se trouvent ces programmes. Par conséquent, nous vous suggérons d'ajouter la ligne suivante au fichier .bashrc dans votre répertoire personnel :

```
PATH=$PATH:/usr/sbin:/sbin
```

Si vous vous déconnectez puis vous reconnectez (vous n'avez pas besoin de redémarrer), cela sera pleinement efficace.

Chapitre 2

Les bases de Kubernetes

2 Laboratoires

Exercice 2.1 : Afficher les ressources en ligne

Avec un projet qui évolue si rapidement, il est important de suivre les mises à jour. L'endroit principal pour trouver la documentation de la version actuelle est https://kubernetes.io/docs/home/.

1. Ouvrez un navigateur et visitez le site web https://kubernetes.io/docs/home/.
2. Dans le coin supérieur droit, utilisez le menu déroulant pour afficher les versions disponibles. On est actuellement à la version v1.29.
3. Sélectionnez le lien de niveau supérieur pour Documentation. Les liens à gauche de la page peuvent être utiles dans la navigation.
4. Si le temps le permet, parcourez d'autres sous-pages telles que CONFIGURATION, CONCEPTS et TÂCHES pour vous familiariser avec le mise en page

Suivre les problèmes de Kubernetes

Des centaines, voire des milliers, travaillent sur Kubernetes chaque jour. Avec autant de personnes travaillant en parallèle, il existe de bonnes ressources pour voir si d'autres connaissent une panne similaire. Le code source ainsi que le suivi des fonctionnalités et des problèmes se trouvent actuellement sur github.com.

1. Pour afficher la page principale, utilisez votre navigateur pour visiter https://github.com/kubernetes/kubernetes/
2. Cliquez sur différents sous-répertoires et affichez les informations de base disponibles.
3. Mettez à jour votre URL vers : https://github.com/kubernetes/kubernetes/issues. Vous

devriez voir une série de problèmes, de demandes de fonctionnalités et de communications d’assistance.

4. Vous devriez maintenant voir les bogues par ordre décroissant. En haut des numéros, une zone de menu vous permet également d'afficher les entrées par auteur, étiquettes, projets, jalons et responsable. Prenez un moment pour consulter les différents autres critères de sélection.

Chapitre 3

Installation et configuration

3 Laboratoires

Exercice 3.1 : Installer Kubernetes

Il existe plusieurs outils d'installation Kubernetes fournis par différents fournisseurs. Dans cet atelier, nous apprendrons à utiliser kubeadm. En tant qu'outil indépendant soutenu par la communauté, il est prévu de devenir le principal moyen de créer un cluster Kubernetes.

Plateformes : Digital Ocean, GCP, EKS de AWS, AKS de Microsoft Azure.. etc.

Les ateliers ont été écrits à l'aide d'instances Ubuntu 20.04 exécutées. Ils ont été écrits pour être indépendants du fournisseur et peuvent donc fonctionner sur n'importe quel plateforme, sur du matériel local ou dans la virtualisation pour vous offrir plus de flexibilité et d'options. Chaque plateforme aura des méthodes d'accès et des considérations différentes. Toute au long de nos atelier on utilisera de la version 1.29.0 de kuberntes.
Si vous utilisez votre propre équipement, vous devrez désactiver le swap sur chaque nœud et vous assurer qu'il n'y a qu'une seule interface réseau.

Plusieurs interfaces sont prises en charge mais nécessitent une configuration supplémentaire. Il peut y avoir d'autres exigences qui seront affichées sous forme d'avertissements ou d'erreurs lors de l'utilisation de la commande **kubeadm** . Bien que la plupart des commandes soient exécutées en tant qu'utilisateur standard, certaines nécessitent les privilèges root. Veuillez configurer l'accès **sudo**. Si vous accédez aux nœuds à distance, par exemple avec GCP ou AWS, vous devrez utiliser un client SSH tel qu'un terminal local **Mobaxterm** ou **PuTTY,** si vous n'utilisez pas Linux ou Mac. Vous auriez également besoin d'un fichier **.pem** ou **.ppk** pour accéder aux nœuds.

Très important

Veuillez désactiver tous les parefeu pendant l'apprentissage de Kubernetes. Bien qu'il existe une liste de ports requis pour la communication entre les composants, cette liste peut ne pas être aussi complète que nécessaire. Si vous utilisez AWS par exemple, vous pouvez ajouter une règle au projet qui autorise tout le trafic vers tous les ports.

Fichiers YAML et espaces blancs

Divers exercices utiliseront des fichiers YAML, inclus dans le texte. Je recommande à écrire certains fichiers histoire de s'entrainer, car la syntaxe de YAML comporte des exigences d'indentation d'espaces blancs qu'il est important d'apprendre. N'utilisez pas de tabulations dans vos fichiers YAML, des espaces blancs seulement

Dans l'exercice suivant, nous pouvons installer Kubernetes sur une seule machine, puis développer le cluster, en ajoutant davantage de ressources de calcul. Les deux nœuds utilisés (master et worker) ont la même taille au départ, fournissant 2 vCPU et 2 Go ou 4Go de mémoire selon vos ressources. Des nœuds plus petits pourraient être utilisés, mais fonctionneraient plus lentement.

Lien Github :
Pour éviter des longues et fastidieuses manipulation afin d'installer kubernetes sur deux nœuds voici un script dans le lien ci-joint utilisant Vagrant pour installer un master et worker :
https://github.com/grassanordine/kubernetes-certification-stack.git

```
etudiant@k8s:~$ sudo -i
root@k8s:~# git clone https://github.com/grassanordine/kubernetes-certification-stack.git
root@k8s:~# cd kubernetes-certification-stack
root@k8s:~# vagrant up
root@k8s:~# vagrant ssh master
[vagrant@master ~]$  sudo su
[root@master vagrant]# kubectl get node
```

```
NAME     STATUS    ROLES          AGE   VERSION
master   Ready     control-plane  68d   v1.28.6
worker1  Ready     <none>         68d   v1.28.6
```

un aperçu sur le fichier **Vagrantfile**

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
  config.vm.define "master" do |master|
    master.vm.box = "geerlingguy/centos7"
    master.vm.network "private_network", type: "static", ip: "192.168.99.10"
    master.vm.hostname = "master"
    master.vm.provider "virtualbox" do |v|
      v.name = "master"
      v.memory = 4096
      v.cpus = 2
    end
    master.vm.provision :shell do |shell|
      shell.path = "install_kubernetes.sh"
      shell.args = ["master", "192.168.99.10"]
    end
  end
  workers=1
  ram_worker=4096
  cpu_worker=2
  (1..workers).each do |i|
    config.vm.define "worker#{i}" do |worker|
      worker.vm.box = "geerlingguy/centos7"
      worker.vm.network "private_network", type: "static", ip: "192.168.99.1#{i}"
      worker.vm.hostname = "worker#{i}"
      worker.vm.provider "virtualbox" do |v|
        v.name = "worker#{i}"
        v.memory = ram_worker
        v.cpus = cpu_worker
      end
      worker.vm.provision :shell do |shell|
        shell.path = "install_kubernetes.sh"
        shell.args = ["node", "192.168.99.10"]
      end
    end
  end
end
```

Figure I : Script pour monter un cluster Kubernetes

Pour plus de détails a propos de la mise en place d'un cluster Kubernetes. Veuillez vous référer au lien : https://github.com/grassanordine/kubernetes-certification-stack.git ou vous trouverez toutes les informations nécessaires pour aller jusqu'au bout de l'installation.

Vous pouvez aussi installé un cluster Kubernetes en allant sur le site officiel de Kubernetes dont un guide dans le lien ci-dessous :
https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/

Mais comme je l'ai déjà mentionné c'est une tâche qui ne sera pas simple, surtout pour un débutant. C'est pour cela que j'ai fait quelques vidéos sur **ma chaine Youtube** qui comporte près de 90 vidéos pour s'y rendre aller sur youtube et taper **@ngrassa.** Dans ces vidéos j'explique l'installation de Kubernetes pour les débutants et les professionnels dans une série de vidéos je vous montre uniquement deux liens mais je vous conseille de voir le reste aussi.

Pour les débutant avec **minikube** :

https://youtu.be/7VkHNzPIHv8?si=C5xj3SkT-Hd6phL7

Pour la production (avec **un master** et **deux workers**) :

https://youtu.be/CNSWcawoIU4?si=n9361MHKtNPyjRCb

Exercice 3.2 : Éléments de post installation

J'ai choisi pour le restant des chapitres de travailler en ligne sur un cluster Kubernetes v1.29 pour que tous les étudiants ou même les intéressés puisse avoir les mêmes résultats que moi.

Le site : https://killercoda.com/playgrounds/scenario/kubernetes

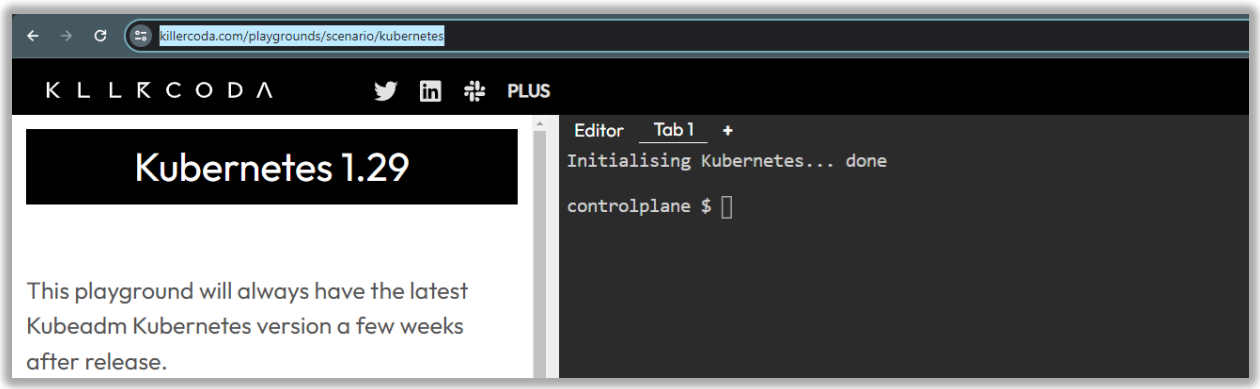

Figure II : Travailler online sur Killercoda

Controlplane$ kubectl get node

```
NAME          STATUS   ROLES          AGE   VERSION
controlplane  Ready    control-plane  35d   v1.29.0
node01        Ready    <none>         35d   v1.29.0
```

Controlplane$ kubeadm config print init-defaults

```yaml
apiVersion: kubeadm.k8s.io/v1beta3
bootstrapTokens:
- groups:
  - system:bootstrappers:kubeadm:default-node-token
  token: abcdef.0123456789abcdef
  ttl: 24h0m0s
  usages:
  - signing
  - authentication
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 1.2.3.4
  bindPort: 6443
nodeRegistration:
  criSocket: unix:///var/run/containerd/containerd.sock
  imagePullPolicy: IfNotPresent
  name: node
  taints: null
---
apiServer:
  timeoutForControlPlane: 4m0s
apiVersion: kubeadm.k8s.io/v1beta3
certificatesDir: /etc/kubernetes/pki
clusterName: kubernetes
controllerManager: {}
dns: {}
etcd:
  local:
    dataDir: /var/lib/etcd
imageRepository: registry.k8s.io
kind: ClusterConfiguration
kubernetesVersion: 1.29.0
networking:
  dnsDomain: cluster.local
  serviceSubnet: 10.96.0.0/12
scheduler: {}
```

Cette configuration définit des paramètres pour initialiser un cluster Kubernetes (**InitConfiguration**) et configurer divers aspects du cluster (**ClusterConfiguration**). Elle comprend des détails tels que les jetons de démarrage, l'endpoint du serveur API, les paramètres d'inscription des nœuds, le délai d'attente du serveur API, le répertoire des certificats, le nom du cluster, le dépôt d'images, la version de Kubernetes, et la configuration réseau. Certaines sections sont laissées vides, indiquant que les paramètres par défaut seront utilisés pour ces composants

Exercice 3.3 : Déployer une application simple

1. Créer un nouveau déploiement nginx et vérifier ensuite l'état « running » et combien de pods ont été lancés.

```
controlplane$ kubectl create deployment nginx --image=nginx
```

```
deployment.apps/nginx created
```

```
controlplane$ kubectl get deployments
```

```
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           21s
```

2. On peut également afficher la sortie au format **yaml**, qui peut être utilisée pour créer à nouveau ce déploiement ou de nouveaux déploiements. On obtient aussi des informations sur l'état courant du déploiement.

```
controlplane$ kubectl get deployment nginx -o yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: "2024-04-07T10:50:12Z"
```

3. Pour afficher les détails du déploiement. Sans oublier que la saisie semi-automatique fonctionnera également pour les sous-commandes et les ressources.

Controlplane$ kubectl describe deployment nginx

```
Name:                   nginx
Namespace:              default
CreationTimestamp:      Sun, 07 Apr 2024 10:50:12 +0000
Labels:                 app=nginx
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=nginx
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:           RollingUpdate
```

4. On va exécuter à nouveau la commande et redirigez la sortie du contenu vers un fichier. Puis modifier ensuite le fichier à sa guise.

   Remarque: Il faut Supprimer **l'horodatage** de création, resourceVersion et lignes uid. A Supprimer également toutes les lignes après status.

controlplane$ kubectl get deployment nginx -o yaml > nginx.yaml

Editer puis supprimer les lignes mentionnées ci-dessus

5. Effacer le déploiement existant

controlplane$ kubectl delete deployment nginx

```
deployment.apps "nginx" deleted
```

6. Créer à nouveau le déploiement cette fois en utilisant le fichier yaml

controlplane$ kubectl create -f nginx.yaml

```
deployment.apps/nginx created
```

7. Maintenant que nous avons travaillé avec le résultat brut, nous allons explorer deux autres façons de générer un YAML ou un JSON utile. En utilisant l' option **--dry-run** et vérifier qu'aucun objet n'a été créé. Seul le déploiement nginx précédent doit être trouvé. La sortie ne contient pas les informations uniques que nous avons supprimées auparavant, mais possède les mêmes valeurs essentielles.

controlplane$ kubectl create deployment two --image=nginx --dry-run=client -o yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: two
  name: two
spec:
<output_omitted>
```

controlplane$ kubectl get deployment

```
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           7m
```

8. Les objets existants peuvent être visualisés dans le format YAML prête à l'emploi. Jetons un coup d' œil au déploiement nginx existent

controlplane$ kubectl get deployments nginx -o yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: null
  generation: 1
  labels:
    run: nginx
<output_omitted>
```

9. La sortie peut également être visualisée dans le format JSON

controlplane$ kubectl get deployment nginx -o json

```
JSON
{
  "apiVersion": "apps/v1",
  "kind": "Deployment",
  "metadata": {
    "annotations": {
      "deployment.kubernetes.io/revision": "1"
    },
"creationTimestamp": "2024-04-07T10:50:12Z",
    .......
  <output_omitted>
```

10. Sachant que nginx est un serveur web, si je tente d'accéder à nginx je dois impérativement exposer un service par-dessus de la pod sur le port 80 bien évidemment. Pour ce faire j'ai deux méthodes l'une impérative en utilisant la ligne des commandes et la deuxième on peut toujours éditer notre fichier nginx.yaml et ajouter le port et protocole manuellement puis forcer de remplacer notre ressource en utilisant l'option --force

```
YAML nginx.yaml
1  ....
2     spec:
3        containers:
4        - image: nginx
5          imagePullPolicy: Always
6          name: nginx
7          ports:                    -- ajout
8          - containerPort: 80       --ajout
9            protocol: TCP           --ajout
10         resources: {}
11 ....
```

controlplane$ kubectl replace -f nginx.yaml --force

```
deployment.apps/nginx replaced
```

11. Affichez le pod et le déploiement. A noter que l'AGE indique que le Pod a été recréé.

controlplane $ kubectl get deploy,pod

```
NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx   1/1     1            1           2m4s

NAME                          READY   STATUS    RESTARTS   AGE
pod/nginx-7db75b8b78-qjffm    1/1     Running   0          8s
```

12. Essayons d'exposer notre ressource. C'est-à-dire créer un service

controlplane $ kubectl expose deployment/nginx

```
service/nginx exposed
```

13. Vérifions la configuration du service. On regarde d'abord le service, puis les informations sur le point de terminaison (endpoints). Notez bien que ClusterIP n'est pas le point de terminaison actuel. Le point de terminaison est fourni par kubelet et kube-proxy. Prenons note de l'adresse IP actuelle du point de terminaison. Dans l'exemple ci-dessous, il s'agit de 192.168.1.5:80. Nous utiliserons ces informations pour accéder à notre pod sur le port 80.

controlplane $ kubectl get svc nginx

```
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
nginx     ClusterIP   10.100.61.122   <none>        80/TCP    3m
```

controlplane $ kubectl get ep nginx

```
NAME    ENDPOINTS         AGE
nginx   192.168.1.5:80    26s
```

14. Pour Tester l'accès à l'adresse IP du cluster sur le port 80. On peut utiliser la commande **curl** en ligne de commande**,** on devrait voir la page générique de nginx installée.

controlplane $ curl 192.168.1.5:80

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
<output_omitted>
```

15. Pour augmenter le nombre de pods on utilise l'option scale, cette option peut réduire ou augmenter le nombre de pods selon le besoin. Voyons d'abord l'état actuel puis faisons travailler la scalabilité.

controlplane $ kubectl get deployment nginx

```
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           12m
```

controlplane $ $ kubectl scale deployment nginx --replicas=3

```
deployment.apps/nginx scaled
```

controlplane $ $ kubectl get deployment nginx

```
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   3/3     3            3           12m
```

16. Affichons à présent les points de terminaison actuels. Il devrait maintenant y en avoir trois. Si la mention À jour ci-dessus (UP-TO-DATE) indique trois, mais disponible (AVAILABLE) indique deux attendez quelques secondes, le déploiement complet pourrait être lent.

controlplane $ kubectl get ep nginx

```
NAME    ENDPOINTS                                       AGE
nginx   192.168.0.3:80,192.168.1.5:80,192.168.1.6:80   7m40s
```

17. On va chercher le pod le plus ancien du déploiement nginx et le supprimer. La touche Tab peut être utile pour les noms longs. Utilisons le Champ AGE pour déterminer lequel est le plus ancien. Puis voyons le comportement de Kubernetes

controlplane $ kubectl get pod -o wide

```
NAME                    READY   STATUS    RESTARTS   AGE   IP
nginx-1423793266-7f1qw  1/1     Running   0          14m   192.168.1.5
nginx-1423793266-8w2nk  1/1     Running   0          86s   192.168.1.6
nginx-1423793266-fbt4b  1/1     Running   0          86s   192.168.0.3
```

controlplane $ kubectl delete pod nginx-1423793266-7f1qw

```
pod "nginx-1423793266-7f1qw" deleted
```

18. Attendons une minute ou deux, puis visualisons à nouveau les pods. L'un être plus récent que les autres. Dans l'exemple suivant, neuf secondes au lieu de quatre minutes. Même le nom de la pod si on regarde bien a été changé.

controlplane $ kubectl get po

```
NAME                    READY   STATUS    RESTARTS   AGE
nginx-1423793266-13p69  1/1     Running   0          9s
nginx-1423793266-8w2nk  1/1     Running   0          4m1s
nginx-1423793266-fbt4b  1/1     Running   0          4m1s
```

controlplane $ $ kubectl get ep nginx

```
NAME    ENDPOINTS                                       AGE
nginx   192.168.0.3:80,192.168.1.6:80,192.168.1.7:80   12m
```

Enfin si on teste à nouveau l'accès au serveur Web, en utilisant l'adresse ClusterIP, puis l'une des adresses IP du point de terminaison. Même si les points de terminaison ont changé, on peut toujours accéder au serveur Web. Cet accès s'effectue uniquement depuis l'intérieur du cluster pour le moment.

Exercice 3.4 : Accès depuis l'extérieur du cluster

On peut accéder à un service depuis l'extérieur du cluster à l'aide d'un module complémentaire DNS comme **coreDNS** ou de variables d'environnement. Nous utiliserons des variables d'environnement pour cet exemple

1. On va recommencer de nouveau par créer un pod puis scaler (augmenter) à 3 le nombre de pods

controlplane $ kubectl create deployment nginx --image=nginx

```
deployment.apps/nginx created
```

controlplane $ kubectl scale deployment nginx --replicas=3

```
deployment.apps/nginx  scaled
```

2. On va créer un service de type LoadBlancer, A noter que le type LoadBalancer est différent du type ClusterIP. SI on vérifie l'état des ports externes. La sortie affichera l'adresse IP externe comme en attente (pending) . À moins qu'un fournisseur ne

réponde avec un équilibreur de charge avec AWS, AZURE …etc. il continuera à apparaître comme en attente. Pour y remédier à ce problème on utilisera un outil qui s'appelle ngrok.

controlplane $ kubectl expose deployment nginx --type=LoadBalancer --port 80

```
deployment.apps/nginx exposed
```

controlplane $ kubectl get svc

```
NAME        TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)      AGE
kubernetes  ClusterIP      10.96.0.1      <none>        443/TCP      15d
nginx       LoadBalancer   10.104.12.173  <pending>     80:30781/TCP 21s
```

3. Installation de l'outil ngrok sous Linux

controlplane $ snap install ngrok

```
ngrok (v3/stable) 3.7.0 from Kyle Wenholz (kyle-ngrok) installed
```

4. Ajoutons notre jeton d'authentification au ngrok.yml par défaut. Pour cela voir le lien ci-après : https://dashboard.ngrok.com/get-started/setup/linux

controlplane $ ngrok config add-authtoken 2bzLyLONGE63g8H8eZT2CKBJXMD_39wkkK8XRDGumiEdbzLf2

```
Authtoken saved to configuration file: /root/snap/ngrok/138/.config/ngrok/ngrok.yml
```

5. Déployons notre application en ligne

controlplane $ ngrok http http://10.104.12.173:80

```
ngrok

K8s Gateway API https://ngrok.com/early-access/kubernetes-gateway-api

Session Status                online
Account                       ngrassa@gmail.com (Plan: Free)
Update                        update available (version 3.8.0, Ctrl-U to update)
Version                       3.7.0
Region                        Europe (eu)
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://eff2-51-89-20-154.ngrok-free.app -> http://10.104.12.173:80

Connections                   ttl     opn     rt1     rt5     p50     p90
                              0       0       0.00    0.00    0.00    0.00
```

Figure III : Utilisation de ngrok pour

6. Une fois exécutés, nos points de terminaison seront répertoriés sur la page des points de terminaison et l'accès à la page web se fait via le lien ci-dessous : https://eff2-51-89-20-154.ngrok-free.app

   Remarque : le DNS fourni est provisoire. Le mieux dans ce cas de passer par les service du cloud tel que AWS, Google Cloud ou mircosoft Azure.

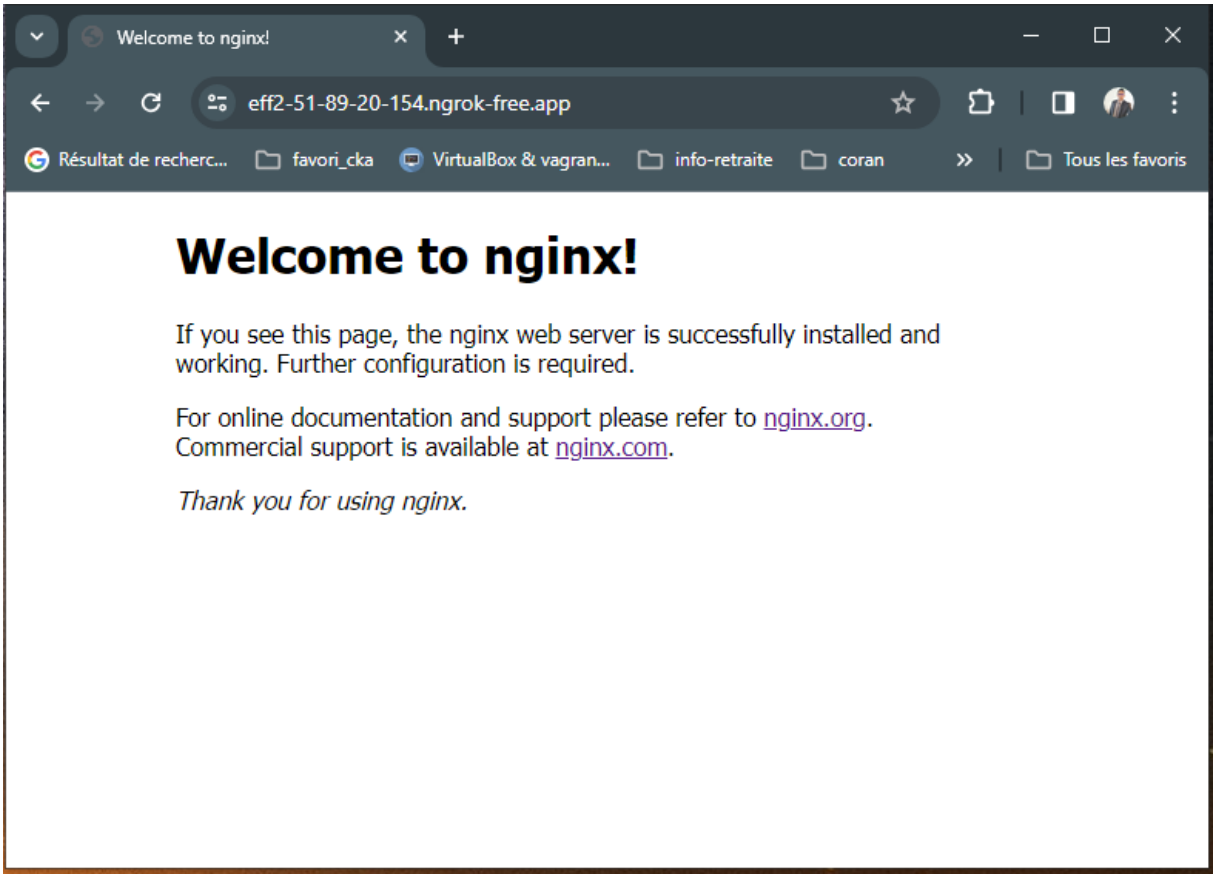

Figure IV : Accès externe via navigateur

Chapitre 4

Architecture de Kubernetes

4 Laboratoires

Conception : Les nœuds de base dans une architecture Kubernetes

1. Source : https://kubernetes.io/docs/concepts/architecture/

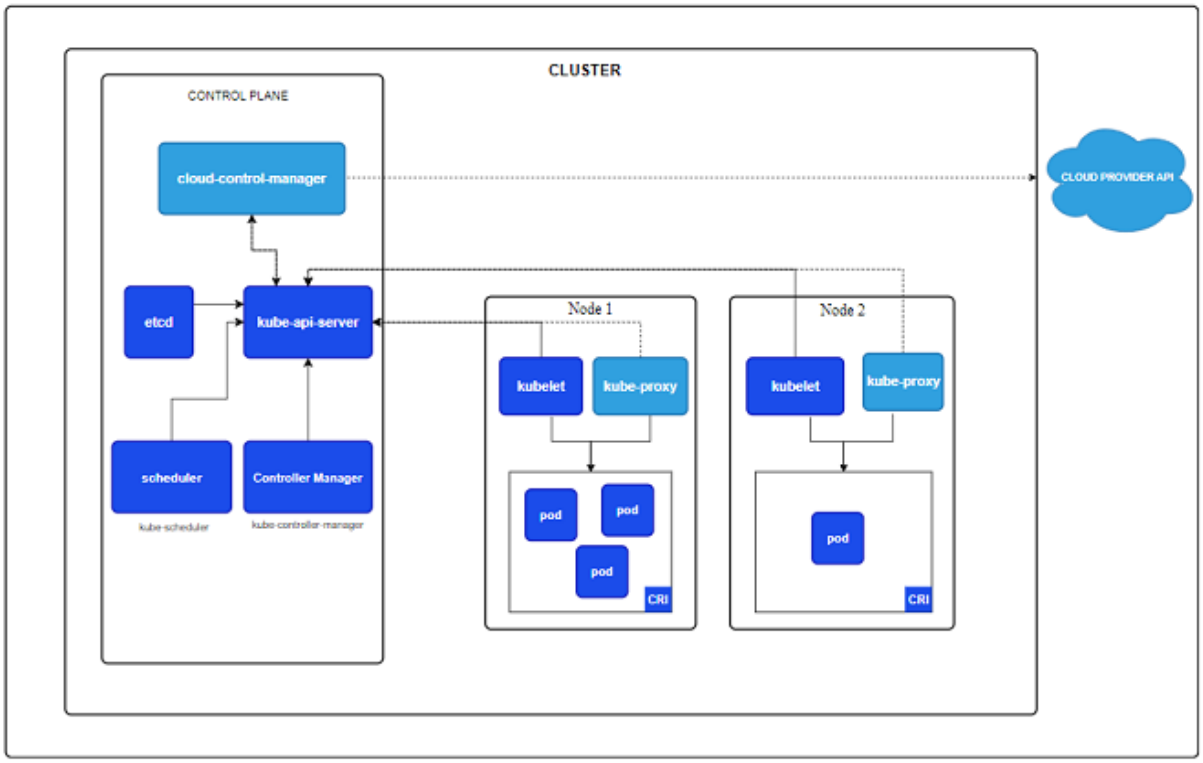

Figure V : Architecture des clusters Kubernetes

Exercice 4.1 : Sauvegarder la base de données etcd

Dans cette section, nous allons sauvegarder la base de données etcd puis mettre à jour la version de Kubernetes utilisée sur les nœuds du plan de contrôle et les nœuds de travail. Il reste toujours judicieux de sauvegarder l'état du cluster avant la mise à niveau. Là il existe de nombreux outils disponibles sur le marché pour sauvegarder et gérer **etcd**, chacun avec un processus de sauvegarde et de restauration distinct. Nous allons utilisez la commande

snapshot, mais a noter que les étapes exactes de restauration dépendront des outils utilisés, de la version du

cluster et la nature de la catastrophe.

1. Je vais chercher d'abord le répertoire de données du démon etcd. Tous les paramètres du pod se trouveront dans le répertoire nommé manifest.

   controlplane $ sudo grep data-dir /etc/kubernetes/manifests/etcd.yaml

```
- --data-dir=/var/lib/etcd
```

2. Accédons dans le pod etcd :

(a) Affichons les arguments et les options de la commande **etcdctl**. Prenons un moment pour visualiser les options et les arguments disponibles. Comme le Bourne shell n'a pas de nombreuses fonctionnalités, il peut être plus facile de copier/coller le majorité de la commande et des arguments après les avoir tapés la première fois.

controlplane $ kubectl -n kube-system exec -it etcd-controlplane -- sh

sh-5.2# etcdctl -h

```
NAME:
        etcdctl - A simple command line client for etcd3.

USAGE:
        etcdctl [flags]
<output_omitted>
```

(b) Pour utiliser TLS, recherchons les trois fichiers qui doivent être transmis avec la commande etcdctl. Il faut accèder au répertoire et affichez les fichiers disponibles. Les versions plus récentes de l'image etcd ont été réduites. Par conséquent on peut ne pas trouver la commande find, et de même la plupart des commandes. Il faut se souvenir du chemin **/etc/kubernetes/pki/etcd**. Comme la commande **ls** est également manquante, nous pouvons plutôt afficher les fichiers en utilisant **echo**.

sh-5.2# cd /etc/kubernetes/pki/etcd/

sh-5.2# echo *

```
ca.crt ca.key healthcheck-client.crt healthcheck-client.key
peer.crt peer.key server.crt server.key
```

(c) La saisie de chacune de ces touches, en particulier dans un shell verrouillé, peut être évitée en utilisant un paramètre environnemental. Déconnectons-nous du shell et faisons-le depuis le master

sh-5.2# exit

3. Vérifions l'état de la base de données à l'aide de l'adresse IP de bouclage 127.0.0.1 et du port 2379. On devrait ensuite transmettre le certificat, la paire de clé ainsi que l'autorité de certification en tant que variables environnementales.

```
controlplane $ kubectl -n kube-system exec -it etcd-controlplane -- sh \
-c "ETCDCTL_API=3 \
ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt \
ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt \
ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key \
etcdctl endpoint health"
```

```
127.0.0.1:2379 is healthy: successfully committed proposal: took = 26.321362ms
```

4. Dans un environnement de production pour prévoir 50%+1 pour le quorum, il faut déterminer combien de bases de données vais-je utilisée pour le cluster. Trois ou cinq sont courants. Dans notre environnement d'exercice actuel, nous ne verrons qu'une seule base de données.

```
controlplane $ kubectl -n kube-system exec -it etcd-cp -- sh \
-c "ETCDCTL_API=3 \
ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt \
ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt \
ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key \
etcdctl --endpoints=https://127.0.0.1:2379 member list"
```

```
264d7b068180479b, started, controlplane, https://172.30.1.2:2380,
https://172.30.1.2:2379, false
```

5. On peut également afficher l'état du cluster sous forme de tableau, entre autres transmis avec l'option -w. La flèche vers le haut nous permet de modifier facilement uniquement la dernière partie de la longue chaîne.

```
controlplane $ kubectl -n kube-system exec -it etcd-controlplane -- sh \
-c "ETCDCTL_API=3 ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt \
```

```
ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt \
ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key \
etcdctl --endpoints=https://127.0.0.1:2379 member list -w table"
```

```
+------------------+---------+--------------+-------------------------+-------------------------+------------+
|        ID        | STATUS  |     NAME     |        PEER ADDRS       |       CLIENT ADDRS      | IS LEARNER |
+------------------+---------+--------------+-------------------------+-------------------------+------------+
| 264d7b068180479b | started | controlplane | https://172.30.1.2:2380 | https://172.30.1.2:2379 |      false |
+------------------+---------+--------------+-------------------------+-------------------------+------------+
```

6. Maintenant que nous savons combien de bases de données etcd se trouvent dans le cluster et leur état de santé, nous pouvons les sauvegarder. L'argument **snapshot** est essentielle pour enregistrer l'instantané dans le répertoire de données du conteneur **/var/lib/etcd/**

```
controlplane $ kubectl -n kube-system exec -it etcd-controlplane -- sh \
-c "ETCDCTL_API=3 \
ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt \
ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt \
ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key \
etcdctl --endpoints=https://127.0.0.1:2379 snapshot save /var/lib/etcd/snapshot.db "
```

```
{"level":"info","ts":"2024-03-20T14:11:32.402345Z","caller":"snapshot/v3_snapshot.go:65","msg":"created temporary db file","path":"/var/lib/etcd/snapshot.db.part"}
{"level":"info","ts":"2024-03-20T14:11:32.448047Z","logger":"client","caller":"v3@v3.5.10/maintenance.go:212","msg":"opened snapshot stream; downloading"}
{"level":"info","ts":"2024-03-20T14:11:32.450381Z","caller":"snapshot/v3_snapshot.go:73","msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info","ts":"2024-03-20T14:11:33.078024Z","logger":"client","caller":"v3@v3.5.10/maintenance.go:220","msg":"completed snapshot read; closing"}
{"level":"info","ts":"2024-03-20T14:11:33.115912Z","caller":"snapshot/v3_snapshot.go:88","msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379","size":"5.9 MB","took":"now"}
{"level":"info","ts":"2024-03-20T14:11:33.116192Z","caller":"snapshot/v3_snapshot.go:97","msg":"saved","path":"/var/lib/etcd/snapshot.db"}
Snapshot saved at /var/lib/etcd/snapshot.db
```

```
controlplane $ sudo ls -l /var/lib/etcd/
```

```
total 3888
drwx------ 4 root root    4096  25 11:22 member
-rw------- 1 root root 3973152  25 18:42 snapshot.db
```

7. Sauvegarder l'instantané ainsi que d'autres informations utilisées pour créer le cluster à la fois localement et sur un autre système au cas où le nœud deviendrait indisponible. N'oublions pas de créer régulièrement des instantanés, peut-être en utilisant une tâche **cron** pour garantir une restauration rapide. Lorsque l'on utilise la restauration par instantané, il est important que la base de données ne soit pas utilisée. Un cluster en haute disponibilité supprimerait et remplacerait le nœud master sans aucune nécessité de restauration.

```
controlplane $ mkdir $HOME/backup
controlplane $ sudo cp /var/lib/etcd/snapshot.db $HOME/backup/snapshot.db-$(date +%m-%d-%y)
controlplane $ sudo cp -r /etc/kubernetes/pki/etcd $HOME/backup/
controlplane $ ls $HOME/backup/
```

```
etcd  snapshot.db-03-20-24
```

8. Toute erreur lors de la restauration peut rendre le cluster inutilisable. Pour tenter une restauration de la base de données après le dernier sauvegarde. Voici un lien qui facilite le processus de restauration.
   Lors de la restauration du cluster, on utilise l'option **--data-dir** pour spécifier dans quel dossier le cluster doit être restauré.

```
ETCDCTL_API=3 etcdctl --data-dir <data-dir-location> snapshot restore snapshot.db
```

Pour plus d'information sur la restauration, cliquez ici :

https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/#restoring-an-etcd-cluster

Mettre à niveau le cluster

controlplane $ kubectl get node

```
NAME          STATUS   ROLES          AGE   VERSION
controlplane   Ready    control-plane   17d   v1.29.0
node01         Ready    <none>          17d   v1.29.0
```

1. Commençons par mettre à jour les métadonnées du package pour APT

```
# Find the latest version in the list.

sudo apt update
sudo apt-cache madison kubeadm
```

2. Voyons les packages disponibles. La liste est longue, remontons vers le haut pour trouver une version récente. L'idée c'est de Choisir la prochaine version complète parmi celle que l'on a sur la liste. Par exemple dans notre cluster, si l'on a la version 1.28.0. on rechercherait la version d'après 1.29.0 et ainsi de suite. En l'occurrence ici la nouvelle release est **1.29.3-1.1**. L'utilisation du terme « **madison** » provient de l'outil Debian appelé madison.

```
kubeadm | 1.29.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.29/deb  Packages
kubeadm | 1.29.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.29/deb  Packages
kubeadm | 1.29.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.29/deb  Packages
kubeadm | 1.29.2-1.1 | https://pkgs.k8s.io/core:/stable:/v1.29/deb  Packages
kubeadm | 1.29.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.29/deb  Packages
kubeadm | 1.29.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.29/deb  Packages
kubeadm | 1.28.8-1.1 | https://pkgs.k8s.io/core:/stable:/v1.28/deb  Packages
```

3. Supprimons d'abord la suspension sur kubeadm puis mettons à jour le package. Dans ce cas de figure la mise à jour vers la version **1.29.3-1.1.**
   Enfin n'oublions pas de conserver à nouveau le package kubeadm pour empêcher sa mise à jour avec d'autres logiciels.

controlplane $ sudo apt-mark unhold kubeadm && \
controlplane $ sudo apt-get update && sudo apt-get install -y kubeadm='1.29.3-1.1'
controlplane $ sudo apt-mark hold kubeadm

```
The following packages will be upgraded:
  kubeadm
1 upgraded, 0 newly installed, 0 to remove and 164 not upgraded.
Need to get 10.1 MB of archives.
After this operation, 102 kB of additional disk space will be used.
Get:1 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.29/deb kubeadm 1.29.3-1.1 [10.1 MB]
Fetched 10.1 MB in 0s (21.4 MB/s)
(Reading database ... 132627 files and directories currently installed.)
Preparing to unpack .../kubeadm_1.29.3-1.1_amd64.deb ...
Unpacking kubeadm (1.29.3-1.1) over (1.29.0-1.1) ...
Setting up kubeadm (1.29.3-1.1) ...
kubeadm set on hold.
```

4. Jetons un coup d'œil sur la version de Kubeadm. Il doit indiquer la nouvelle version que nous venons d'installer.

controlplane $ sudo kubeadm version

```
kubeadm version: &version.Info{Major:"1", Minor:"29", GitVersion:"v1.29.3", GitCommit:"6813625b7cd706db5bc7388921be03071e1a492d", GitTreeState:"clean", BuildDate:"2024-03-15T00:06:16Z", GoVersion:"go1.21.8", Compiler:"gc", Platform:"linux/amd64"}
```

5. Pour mettre à jour le nœud master (ici controlplane), nous devons d'abord migrer autant de pods que possible.comme calico et autres pods statique. On peut forcer la migration de quelques pods en ajoutons **--ignore-daemonsets** Il peut aussi nous demander de se détacher des disques durs aussi.

controlplane $ kubectl drain controlplane --ignore-daemonsets

```
node/controlplane cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/canal-j6pwg, kube-system/kube-proxy-85drq
evicting pod local-path-storage/local-path-provisioner-5d854bc5c4-h55kw
evicting pod kube-system/calico-kube-controllers-9d57d8f49-jvt8w
pod/calico-kube-controllers-9d57d8f49-jvt8w evicted
pod/local-path-provisioner-5d854bc5c4-h55kw evicted
node/controlplane drained
```

6. On utilise l'option **upgrade plan** pour vérifier le cluster existant, puis passer à la mise à jour . On remarque que la version installé de **kubeadm** sur le cluster est 1.29.0. On veut passer à présent à la version 1.29.3.

controlplane $ sudo kubeadm upgrade plan

```
[upgrade/config] Making sure the configuration is correct:
[upgrade/config] Reading configuration from the cluster...
[upgrade/config] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[preflight] Running pre-flight checks.
[upgrade] Running cluster health checks
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: v1.29.0
[upgrade/versions] kubeadm version: v1.29.3
[upgrade/versions] Target version: v1.29.3
[upgrade/versions] Latest version in the v1.29 series: v1.29.3
```

7. Nous sommes maintenant prêts à mettre à niveau le logiciel. La commande va nous demander si l'on souhaite procéder à la mise à niveau, on répond par y pour oui. Prenons un moment pour rechercher des erreurs ou des suggestions, telles que la mise à niveau de la version d'etcd ou d'un autre package. Encore une fois, utilisez la prochaine version mineure, la mise à jour à partir de la version utilisée dans l'étape d'avant qui a initialisé le cluster. Ne vous inquiéter pas le processus peut prendre plusieurs minutes.

controlplane $ kubeadm upgrade apply v1.29.3

```
[upgrade/config] Making sure the configuration is correct:
[upgrade/config] Reading configuration from the cluster...
[upgrade/config] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[preflight] Running pre-flight checks.
[upgrade] Running cluster health checks
[upgrade/version] You have chosen to change the cluster version to "v1.29.3"
[upgrade/versions] Cluster version: v1.29.0
[upgrade/versions] kubeadm version: v1.29.3
[upgrade] Are you sure you want to proceed? [y/N]: y
[upgrade/prepull] Pulling images required for setting up a Kubernetes cluster
.....
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.29.3". Enjoy!
```

8. Libérons à présent la suspension sur **kubelet** et **kubectl** et mettons à niveau les deux packages vers la même version que kubeadm. Et n'oublions pas à la fin d'ajouter à nouveau la suspension afin que les autres mises à jour ne mettent pas à jour le logiciel Kubernetes.

```
controlplane $ sudo apt-mark unhold kubelet kubectl
controlplane $ sudo apt-get install -y kubelet=1.29.3-1.1 kubectl=1.29.3-1.1
```

```
Unpacking kubelet (1.29.3-1.1) over (1.29.0-1.1) ...
Setting up kubectl (1.29.3-1.1) ...
Setting up kubelet (1.29.3-1.1) ...
```

```
controlplane $ sudo apt-mark hold kubelet kubectl
```

```
kubelet set on hold.
kubectl set on hold.
```

9. Redémarrer les démons

```
controlplane $ sudo systemctl daemon-reload
controlplane $ sudo systemctl restart kubelet
controlplane $ kubectl get nodes
```

10. Si on vérifie l'état des nœuds. Le master devrait afficher la planification désactivée sur le master.

```
NAME                    STATUS                    ROLES        AGE   VERSION
controlplane   NotReady,SchedulingDisabled   control-plane   17d   v1.29.3
node01         Ready                             <none>       17d   v1.29.0
```

11. On va remettre le status du master en mode Ready et désactivé le verrouillage en même temps.

```
controlplane $ kubectl uncordon controlplane
```

```
node/controlplane uncordoned
```

12. Si on vérifie de nouveau après avoir déverrouillé l'état des nœuds avec l'option **uncordon**. Le master devrait afficher le status Ready.

controlplane $ kubectl get nodes

```
NAME          STATUS   ROLES          AGE   VERSION
controlplane   Ready    control-plane   17d    v1.29.3
node01         Ready    <none>          17d    v1.29.0
```

Enfin on peut mettre à jour la version du noeud1 ou du worker en suivant les mêmes étapes et au même titre que le master (controlplane). Aller d'abord en ssh sur le noeud en question puis commencer à faire les même manipulation. Pour plus d'information cliquez ici :

https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/#drain-the-node

Chapitre 5

Gestion d'état avec les déploiements

5 Laboratoires

Exercice 5.1 : Travailler avec des ReplicaSets

Aperçu

Le but est de comprendre et gérer l'état des conteneurs.c'est une tâche indisponsable de Kubernetes. Dans cet atelier, nous allons d'abord explorer l'API objets utilisés pour gérer des groupes de conteneurs. Les objets disponibles changent à mesure que Kubernetes a mûri, donc la version de Kubernetes utilisée déterminera lesquels sont disponibles. Notre premier objet sera un **ReplicaSet**, qui n'inclut pas les nouvelles fonctionnalités de gestion trouvées avec les déploiements. Un opérateur de déploiement gère ReplicaSet opérateurs pour vous. Nous travaillerons également avec un autre objet et une boucle de surveillance appelée **DaemonSet** qui garantit qu'un conteneur s'exécute sur le nœud nouvellement ajouté. Ensuite, nous mettrons à jour le logiciel dans un conteneur, afficherons enfin l'historique des révisions et reviendrons à une version précédente.

Le but d'un **ReplicaSet** est de maintenir un ensemble stable de pods répliqués en cours d'exécution à tout moment. Il est souvent utilisé pour garantir la disponibilité d'un nombre spécifié de Pods identiques.

1. On va créer un fichier YAML pour un ReplicaSet simple. Le paramètre **apiVersion** dépend de la version de Kubernetes que nous allons utiliser. L'objet est stable en utilisant l'apiVersion **apps/v1**. Nous allons utiliser d'abord une ancienne version de nginx puis mettrons à jour cette version vers une version plus récente plus tard dans l'exercice

```
controlplane $ vi rs.yaml
```

rs.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-one
spec:
  replicas: 2
  selector:
    matchLabels:
      system: ReplicaOne
  template:
    metadata:
      labels:
        system: ReplicaOne
    spec:
      containers:
      - name: nginx
        image: nginx:1.15.1
        ports:
        - containerPort: 80
```

2. Creation d'un réplicaSet

   controlplane $ kubectl create -f rs.yaml

```
replicaset.apps/rs-one created
```

3. Pour afficher tous les ReplicaSets actuels

controlplane $ kubectl get rs

```
NAME     DESIRED   CURRENT   READY   AGE
rs-one     2         2         2      27s
```

4. On remarque dans describe les ReplicaSets nouvellement créés

   controlplane $ kubectl describe rs rs-one

```
Name:         rs-one
Namespace:    default
Selector:     system=ReplicaOne
Labels:       <none>
Annotations:  <none>
Replicas:     2 current / 2 desired
Pods Status:  2 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  system=ReplicaOne
  Containers:
   nginx:
    Image:        nginx:1.15.1
    Port:         80/TCP
…..<output_omitted>
  Normal  SuccessfulCreate  4m7s  replicaset-controller  Created pod: rs-one-nj8g5
  Normal  SuccessfulCreate  4m7s  replicaset-controller  Created pod: rs-one-tf4hk
```

5. Nous voyons ici les pods créés avec le ReplicaSet. À partir du fichier yaml déjà créé, il devrait y avoir deux pods.

controlplane $ kubectl get pods

```
NAME          READY   STATUS    RESTARTS   AGE
rs-one-pwmrq   1/1     Running     0        8m58s
rs-one-tfm75   1/1     Running     0        8m58s
```

6. Nous allons maintenant supprimer le ReplicaSet, mais pas les Pods qu'il contrôle. Autrement dit détaché les pods du ReplicaSet.

controlplane $ kubectl delete rs rs-one --cascade=orphan

```
replicaset.apps "rs-one" deleted
```

7. Affichons à nouveau le ReplicaSet et les pods

controlplane $ kubectl describe rs rs-one

```
Error from server (NotFound): replicasets.apps "rs-one" not found
```

controlplane $ kubectl get pods

```
NAME          READY   STATUS    RESTARTS   AGE
rs-one-pwmrq   1/1     Running     0        36s
rs-one-tfm75   1/1     Running     0        36s
```

8. Créons à nouveau le ReplicaSet. Tant que nous ne modifions pas le champ de sélection, le nouveau ReplicaSet devrait prendre la possession des pods crées précédemment.

controlplane $ kubectl create -f rs.yaml

```
replicaset.apps/rs-one created
```

9. Affichons l'âge du ReplicaSet, puis des pods qu'il contient pour en être sure.

controlplane $ kubectl get rs

```
NAME     DESIRED   CURRENT   READY   AGE
rs-one     2         2         2      9s
```

controlplane $ kubectl get pods

```
NAME          READY   STATUS    RESTARTS   AGE
rs-one-pwmrq   1/1     Running     0          70s
rs-one-tfm75   1/1     Running     0          70s
```

10. Nous allons maintenant isoler un Pod de son ReplicaSet. On commence par modifier le **label** d'un Pod. Nous allons changer le label **system**: pour qu'il soit un pod isolé (**IsolatedPod**). A changer au lieu de **ReplicaOne**

controlplane $ kubectl edit pod rs-one-pwmrq

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cni.projectcalico.org/containerID: 8384cdb7b65197308db2be0cea580e4096
    cni.projectcalico.org/podIP: 192.168.1.4/32
    cni.projectcalico.org/podIPs: 192.168.1.4/32
  creationTimestamp: "2024-03-23T12:55:47Z"
  generateName: rs-one-
  labels:
    system: IsolatedPod
  name: rs-one-pwmrq
  namespace: default
  ownerReferences:
  - apiVersion: apps/v1
    blockOwnerDeletion: true
    controller: true
    kind: ReplicaSet
    name: rs-one
    uid: 965b2a59-2ccd-421a-8f44-0825a67a4fd2
  resourceVersion: "3840"
  uid: 94037a29-6bf9-4e24-9ff6-6a9efaf340a5
spec:
```

```
pod/rs-one-pwmrq edited
```

11. Affichons le nombre de pods dans le ReplicaSet. Vous devriez en voir deux.

controlplane $ kubectl get rs

```
NAME     DESIRED   CURRENT   READY   AGE
rs-one     2         2         2     8m44s
```

12. Affichons maintenant les pods avec la clé d'étiquette du système. Notons qu'il y en a trois, l'un étant plus récent que les autres. Le ReplicaSet a veillé à conserver deux répliques, en remplaçant le Pod qui a été isolé.

```
controlplane $ kubectl get po -L system
```

```
NAME          READY   STATUS    RESTARTS   AGE     SYSTEM
rs-one-pwmrq  1/1     Running   0          9m46s   IsolatedPod
rs-one-tfm75  1/1     Running   0          9m46s   ReplicaOne
rs-one-v4n7h  1/1     Running   0          5m19s   ReplicaOne
```

13. Supprimons le ReplicaSet, puis affichons tous les pods restants, on remarquera qu'il va rester uniquement un seul Pod.

```
controlplane $ kubectl delete rs rs-one
```

```
replicaset.apps "rs-one" deleted
```

```
controlplane $ kubectl get po
```

```
NAME          READY   STATUS    RESTARTS   AGE
rs-one-pwmrq  1/1     Running   0          10m
```

14. Supprimons enfin le Pod restant à l'aide de son étiquette

```
controlplane $ kubectl delete pod -l system=IsolatedPod
controlplane $ kubectl get rs
```

```
No resources found in default namespace.
```

```
controlplane $ kubectl get pod
```

```
No resources found in default namespace.
```

Exercice 5.2 : Utilisation des déploiements

Un déploiement est un objet de boucle de surveillance avec lequel nous avons travaillé déja dans les ateliers précédents. Un déploiement fournit une mise à jour **déclarative** des pods et des **ReplicaSets** et garanti un certain nombre de pods créés en général, plusieurs pouvant se trouver sur un seul nœud. Le déploiement est un objet de ressource de haut niveau utilisé pour gérer le déploiement et la mise à l'échelle des applications conteneurisées.

Un déploiement décrit l'état souhaité de l'application, tel que le nombre de réplicas et l'image du conteneur à utiliser. Lorsqu'un déploiement est créé, Kubernetes créera et gérera automatiquement les réplicas nécessaires, qui à leur tour créeront et gèreront les pods nécessaires pour garantir que l'état souhaité de l'application est atteint. Le déploiement fournit également des mises à jour progressives, qui permettent de mettre à jour une application vers une nouvelle version en minimisant le temps d'arrêt en remplaçant progressivement les anciennes répliques par des nouvelles. L'utilisation de déploiement dans Kubernetes facilite la gestion et la mise à l'échelle des applications conteneurisées tout en garantissant une haute disponibilité et une très grande fiabilité.

1. commençons par créer un fichier yaml en utilisant la méthode impérative. Dans ce cas, le type serait défini sur déploiement. On utilise l'option **–dry-run=client** pour générer son fichier yaml. On pourrait ensuite l'éditer et porter des modification si l'on souhaite.

```
controlplane $ kubectl create deploy webserver --image nginx:1.22.1 --replicas=2 \
--dry-run=client -o yaml  >  dep.yaml

controlplane $  cat dep.yaml
```

dep.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: webserver
  name: webserver
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webserver
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: webserver
    spec:
      containers:
      - image: nginx:1.22.1
        name: nginx
        resources: {}
status: {}
```

controlplane $ kubectl create -f dep.yaml

```
deployment.apps/webserver created
```

controlplane $ kubectl get deploy

```
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
webserver   2/2     2            2           10s
```

controlplane $ kubectl get pod

```
NAME                         READY   STATUS    RESTARTS   AGE
webserver-5fbfb6dcdb-6rncj   1/1     Running   0          21s
webserver-5fbfb6dcdb-f4rgs   1/1     Running   0          21s
```

controlplane $ kubectl describe pod webserver-5fbfb6dcdb-6rncj | grep Image:

```
    Image:          nginx:1.22.1
```

Exercice 5.3 : Mises à jour et restaurations à l'aide du déploiement

L'un des avantages des micro-services est la possibilité de remplacer et de faire évoluer un conteneur tout en continuant à répondre aux attentes du client.

Nous utiliserons le paramètre de recréation qui met à niveau un conteneur lorsque le prédécesseur est supprimé, puis utiliserons la fonctionnalité **RollingUpdate** également, qui démarre immédiatement une mise à jour continue.

1. Commençons d'abord par afficher le paramètre de stratégie actuel pour le déploiement en cours d'utilisation :

controlplane $ kubectl get deploy webserver -o yaml | grep -A 4 strategy

```
strategy:
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 25%
        type: RollingUpdate
```

2. Mettons à jour le déploiement pour utiliser une version plus récente du serveur nginx. Cette fois, on utilise la commande **set** image. La version choisi est **1.23.1-alpine**.

controlplane $ kubectl set image deploy webserver nginx=nginx:1.23.1-alpine

```
deployment.apps/webserver image updated
```

controlplane $ kubectl get pod

```
NAME                          READY   STATUS    RESTARTS   AGE
webserver-7d4f854fdc-75xdf     1/1     Running   0          40s
webserver-7d4f854fdc-bscqz     1/1     Running   0          43s
```

3. Vérifions que le paramètre Image, pour en être sure qu'il a été mise à jour

controlplane $ kubectl describe po webserver-7d4f854fdc-75xdf |grep Image:

```
  Image:          nginx:1.23.1-alpine
```

4. Voici l'affichage de l'historique des modifications pour le déploiement. On voit bien deux révisions répertoriées. Comme nous n'avons pas ajouté l'annotation de cause du changement, nous n'avons pas vu pourquoi l'objet a été mis à jour.

controlplane $ kubectl rollout history deploy webserver

```
deployment.apps/webserver
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

5. Affichons les paramètres des différentes versions du déploiement. La ligne Image: doit être la seule différence entre les deux sorties.

controlplane $ kubectl rollout history deploy webserver --revision=1

```
deployment.apps/webserver with revision #1
Pod Template:
  Labels:       app=webserver
        pod-template-hash=5fbfb6dcdb
  Containers:
   nginx:
    Image:      nginx:1.22.1
    Port:       <none>
    Host Port:  <none>
   Environment:       <none>
    Mounts:     <none>
  Volumes:      <none>
```

controlplane $ kubectl rollout history deploy webserver --revision=2

```
deployment.apps/webserver with revision #2
Pod Template:
  Labels:       app=webserver
        pod-template-hash=7d4f854fdc
  Containers:
   nginx:
    Image:      nginx:1.23.1-alpine
    Port:       <none>
    Host Port:  <none>
```

6. Utilisons Kubectl Rollout Undo pour rétablir le déploiement à la version précédente

controlplane $ kubectl rollout undo deploy webserver

```
deployment.apps/webserver rolled back
```

controlplane $ kubectl get pod

```
NAME                          READY   STATUS    RESTARTS   AGE
webserver-5fbfb6dcdb-5xscw    1/1     Running   0          18s
webserver-5fbfb6dcdb-8shpf    1/1     Running   0          17s
```

controlplane $ kubectl describe pod webserver-5fbfb6dcdb-5xscw | grep Image:

```
  Image:          nginx:1.22.1
```

7. Faisons un peu de ménage en supprimant le fichier de déploiement

controlplane $ kubectl delete deploy webserver

```
deployment.apps "webserver" deleted
```

Exercice 5.4 : Travailler avec les DaemonSets

Un DaemonSet garantit que tous (ou certains) nœuds exécutent une copie d'un pod. Au fur et à mesure que des nœuds sont ajoutés au cluster, des pods leur sont ajoutés. À mesure que les nœuds sont supprimés du cluster, ces pods sont récupérés. La suppression d'un DaemonSet nettoiera les pods qu'il a créés.

Certaines utilisations typiques d'un DaemonSet sont :

- ✓ exécuter un démon de stockage de cluster sur chaque nœud
- ✓ exécuter un démon de collecte de journaux sur chaque nœud
- ✓ exécuter un démon de surveillance de nœud sur chaque nœud

Dans un cas simple, un DaemonSet, couvrant tous les nœuds, serait utilisé pour chaque type de démon. Une configuration plus complexe peut utiliser plusieurs DaemonSets pour un seul type de démon, mais avec des indicateurs différents et/ou des demandes de mémoire et de processeur différentes pour différents types de matériel.

1. Nous commençons par créer un fichier yaml. Dans ce cas, l'objet serait défini sur DaemonSet.

controlplane $ vi ds-one.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ds-one
  labels:
    app: nginx
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.15.1
        ports:
        - containerPort: 80
          name: http
        volumeMounts:
        - name: html
          mountPath: /usr/share/nginx/html
      volumes:
      - name: html
        hostPath:
          path: /var/www/html
```

2. Créons et vérifions aussi le DaemonSet nouvellement créer. Il doit y avoir un pod dans chacun des nœuds du cluster.

controlplane $ kubectl apply -f ds-one.yaml

```
daemonset.apps/ds-one created
```

controlplane $ kubectl get ds

```
NAME    DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
ds-one    2         2         1        2            1          <none>         8s
```

controlplane $ kubectl get po -o wide

```
NAME          READY  STATUS   RESTARTS  AGE  IP            NODE         NOMINATED NODE  READINESS GATES
ds-one-4t7lx   1/1   Running    0        20s  192.168.1.4  node01            <none>        <none>
ds-one-pd5x6   1/1   Running    0        20s  192.168.0.4  controlplane      <none>        <none>
```

3. On va vérifier l'image exécutée à l'intérieur des pods. Nous aurons de ces informations dans la section suivante.

controlplane $ kubectl describe pod ds-one-4t7lx | grep Image:

```
Image:        nginx:1.15.1
```

Exercice 5.5 : Mises à jour et restaurations progressives à l'aide de DaemonSet

L'un des avantages des micro-services est la possibilité de remplacer et de faire évoluer un conteneur tout en continuant à répondre aux attentes du client.
Nous utiliserons le paramètre **OnDelete** qui met à niveau un conteneur lorsque le prédécesseur est supprimé, puis utiliserons la fonctionnalité **RollingUpdate** également, qui démarre immédiatement une mise à jour continue

1. Commençons par afficher le paramètre **updateStrategy** actuel pour le **DaemonSet** créé dans la section précédente.

controlplane $ kubectl get ds ds-one -o yaml | grep -A 4 Strategy

```
updateStrategy:
  rollingUpdate:
    maxSurge: 0
    maxUnavailable: 1
  type: RollingUpdate     # Editer et modifier le type  par OnDelete
```

controlplane $ kubectl get ds ds-one -o yaml > ds-one1.yaml
controlplane $ kubectl replace --force -f ds-one1.yaml

```
daemonset.apps "ds-one" deleted
daemonset.apps/ds-one replaced
```

controlplane $ kubectl get po

```
NAME          READY  STATUS   RESTARTS  AGE
ds-one-5rs8h   1/1   Running    0       13s
ds-one-fjr5n   1/1   Running    0       13s
```

2. Mettons à jour DaemonSet pour utiliser une version plus récente du serveur nginx. Toujours en mode impérative on passe à la version **1.16.1-alpine**.

controlplane $ kubectl set image ds ds-one nginx=nginx:1.16.1-alpine

```
daemonset.apps/ds-one image updated
```

3. Vérifions que le paramètre Image: du Pod coché dans la section précédente est inchangé.

controlplane $ kubectl describe po ds-one-5rs8h |grep Image:

```
Image:        nginx:1.15.1
```

4. Supprimons le pod. Et Attendons que le Pod de remplacement prend effet puis vérifions de nouveau la version

controlplane $ kubectl delete po ds-one-5rs8h

```
pod "ds-one-5rs8h" deleted
```

controlplane $ kubectl get po

```
NAME          READY   STATUS    RESTARTS   AGE
ds-one-fjr5n   1/1     Running   0          103s
ds-one-nl5bj   1/1     Running   0          5s
```

5. Affichons l'image exécutée sur l'ancien pod. Il devrait toujours afficher la version **1.15.1**

controlplane $ kubectl describe po ds-one-fjr5n |grep Image:

```
Image:        nginx:1.15.1
```

6. Affichons l'image exécutée sur le nouveau pod. Il devrait afficher la version **1.16.1-alpine**

controlplane $ kubectl describe po ds-one-nl5bj |grep Image:

```
Image:        nginx:1.16.1-alpine
```

7. Affichons l'historique des modifications pour le DaemonSet. On peut voir deux révisions répertoriées. Comme nous n'avons pas ajouté l'annotation de cause du changement, alors nous verrons pas pourquoi l'objet a été mis à jour.

controlplane $ kubectl rollout history ds ds-one

```
daemonset.apps/ds-one
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

8. Affichons à présent les paramètres des différentes versions de DaemonSet. La ligne Image: devrait être la seule différence entre les deux sorties

controlplane $ kubectl rollout history ds ds-one --revision=1

```
daemonset.apps/ds-one with revision #1
Pod Template:
 Labels:     app=nginx
 Containers:
  nginx:
   Image:     nginx:1.15.1
   Port:      80/TCP
   Host Port:  0/TCP
   Environment:      <none>
   Mounts:
    /usr/share/nginx/html from html (rw)
 Volumes:
  html:
   Type:      HostPath (bare host directory volume)
   Path:      /var/www/html
   HostPathType:
```

controlplane $ kubectl rollout history ds ds-one --revision=2

```
daemonset.apps/ds-one with revision #2
Pod Template:
 Labels:     app=nginx
 Containers:
  nginx:
   Image:     nginx:1.16.1-alpine
   Port:      80/TCP
   Host Port:  0/TCP
   Environment:      <none>
   Mounts:
.........etc.
```

9. Créons un nouveau DaemonSet, en définissant cette fois la stratégie de mise à jour sur **RollingUpdate** et non pas **OnDelete**. Commencons par générer un nouveau fichier de configuration.

controlplane $ kubectl get ds ds-one -o yaml > ds2.yaml
controlplane $ vim ds2.yaml

```
....
  name: ds-two
....
    type: RollingUpdate
```

controlplane $ kubectl create -f ds2.yaml

```
daemonset.apps/ds-two created
```

10. Vérifions que les Pods utilisent la nouvelle version du logiciel

controlplane $ kubectl get pod

```
NAME        READY   STATUS    RESTARTS   AGE
ds-one-fjr5n    1/1    Running   0        9m42s
ds-one-nl5bj    1/1    Running   0        8m4s
ds-two-8sh57    1/1    Running   0        12s
ds-two-brqzs    1/1    Running   0        12s
```

controlplane $ kubectl describe po ds-two-8sh57 |grep Image:

```
Image:        nginx:1.16.1-alpine
```

controlplane $ kubectl describe po ds-two-brqzs |grep Image:

```
Image:        nginx:1.16.1-alpine
```

controlplane $ kubectl rollout status ds ds-two

```
daemon set "ds-two" successfully rolled out
```

11. Vérifions tous les Pods. Que remarquez-vous ?

controlplane $ kubectl get pod

```
NAME        READY   STATUS    RESTARTS   AGE
ds-one-fjr5n   1/1    Running     0          12m
ds-one-nl5bj   1/1    Running     0          10m
ds-two-8sh57   1/1    Running     0          2m33s
ds-two-brqzs   1/1    Running     0          2m33s
```

controlplane $ kubectl describe po ds-two-8sh57 |grep Image:

```
Image:        nginx:1.16.1-alpine
```

controlplane $ kubectl describe po ds-two-brqzs |grep Image:

```
Image:        nginx:1.16.1-alpine
```

controlplane $ kubectl describe po ds-one-nl5bj |grep Image:

```
Image:        nginx:1.16.1-alpine
```

controlplane $ kubectl describe po ds-one-fjr5n |grep Image:

```
Image:        nginx: 1.15.1
```

12. Enfin Nettoyons le système en supprimant les DaemonSets

controlplane $ kubectl delete ds ds-one ds-two

```
daemonset.apps "ds-one" deleted
daemonset.apps "ds-two" deleted
```

Chapitre 6

Volumes et données

6 Laboratoires

Exercice 6.1 : Créer une ConfigMap

Un ConfigMap est un objet API utilisé pour stocker des données non confidentielles dans des paires clé-valeur. Les pods peuvent utiliser des ConfigMaps en tant que variables d'environnement, arguments de ligne de commande ou en tant que fichiers de configuration dans un volume. Ces données peuvent être mises à disposition afin qu'un pod puisse lire les données en tant que variables d'environnement ou données de configuration.

Attention : ConfigMap ne fournit ni secret ni cryptage. Si les données que vous souhaitez stocker sont confidentielles, utilisez un secret plutôt qu'un ConfigMap, ou utilisez des outils supplémentaires (tiers) pour garder vos données privées.

1. Nous allons maintenant créer un ConfigMap et le remplir avec des clés et des valeurs littérale avec la ligne de commande.

controlplane $ kubectl create configmap colors --from-literal=app1=black --from-literal=app2=yello --from-literal=app3=blue

```
configmap/colors created
```

2. Visualisez comment les données sont organisées à l'intérieur du cluster en format yaml.

controlplane $ kubectl get configmap colors

```
NAME     DATA   AGE
colors     3     18s
```

controlplane $ kubectl get configmap colors -o yaml

```
apiVersion: v1
data:
  app1: black
  app2: yello
  app3: blue
kind: ConfigMap
metadata:
  creationTimestamp: "2024-03-25T10:26:42Z"
  name: colors  …….etc.
```

3. Nous pouvons maintenant créer un Pod pour utiliser le ConfigMap. Dans ce cas, un paramètre particulier **env** est défini comme variable d'environnement.

controlplane $ vim simpleshell.yaml

simpleshell.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: test
      valueFrom:
        configMapKeyRef:
          name: colors
          key: app1
```

controlplane $ kubectl create -f simpleshell.yaml

```
pod/demo created
```

4. Créons le pod et affichons d'abord la variable d'environnement **app1** puis **app2**.

controlplane $ kubectl exec demo -- /bin/bash -c 'echo $test'

black

controlplane $ vim simpleshell.yaml

```yaml
- name: test
  valueFrom:
    configMapKeyRef:
      name: colors
      key: app2    ---> ligne à modifier
```

controlplane $ kubectl replace --force -f simpleshell.yaml

```
pod "demo" deleted
pod/demo replaced
```

controlplane $ kubectl exec demo -- /bin/bash -c 'echo $test'

yello

controlplane $ kubectl delete pod demo

```
pod "demo" deleted
```

5. Toutes les variables d'un fichier peuvent également être incluses en tant que variables d'environnement. Ajoutons pour cela **envFrom** et **configMapRef** en changant le code du fichier précèdent. Recréons le pod à nouveau.

controlplane $ vim simpleshell.yaml

simpleshell.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: colors
```

controlplane $ kubectl create -f simpleshell.yaml

```
pod/demo created
```

controlplane $ kubectl exec demo -- /bin/bash -c 'env'

```
KUBERNETES_SERVICE_PORT_HTTPS=443
app2=yello
app3=blue
app1=black
KUBERNETES_SERVICE_PORT=443
HOSTNAME=demo
<output_omitted>
```

6. Un ConfigMap peut également être créé à partir d'un fichier YAML. Créons-en un avec quelques paramètres pour décrire une voiture.

controlplane $ vim car-map.yaml

car-map.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: fast-car
  namespace: default
data:
  car.make: Ford
  car.model: Mustang
  car.trim: Shelby
```

7. Créons le ConfigMap et vérifions ses paramètres.

controlplane $ kubectl create -f car-map.yaml

```
configmap/fast-car created
```

controlplane $ kubectl get configmap fast-car -o yaml

```
apiVersion: v1
data:
  car.make: Ford
  car.model: Mustang
  car.trim: Shelby
kind: ConfigMap
metadata:
  creationTimestamp: "2024-03-25T15:52:45Z"
  name: fast-car
  namespace: default
  resourceVersion: "2770"
  uid: 13f79386-716a-4596-b2e9-8c06b8278b93
```

8. Nous allons maintenant rendre le ConfigMap disponible pour un Pod en tant que volume monté. Nous pouvons réutilisé le fichier précédent et ajouter les entrées suivantes pour notre conteneur tel que **volumes** et **configMap**.

controlplane $ vim simpleshell.yaml

simpleshell.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: car-vol
          mountPath: /etc/cars
  volumes:
    - name: car-vol
      configMap:
        name: fast-car
```

9. Créons à nouveau le pod et surtout Vérifions que le volume existe ainsi que le contenu du fichier.

```
controlplane $ kubectl create -f simpleshell.yaml
```

```
configmap/demo created
```

```
controlplane $ kubectl exec demo -- /bin/bash -c 'df -ha |grep car'
```

```
/dev/vda1      20G  5.9G   14G  31% /etc/cars
```

```
controlplane $ kubectl exec demo -- /bin/bash -c 'ls /etc/cars/'
```

```
car.make
car.model
car.trim
```

```
controlplane $ kubectl exec demo -- /bin/bash -c 'cat /etc/cars/car.trim'
Shelby

controlplane $ kubectl exec demo -- /bin/bash -c 'cat /etc/cars/car.model'
Mustang

controlplane $  kubectl delete pods demo
```

```
pod "demo" deleted
```

La gestion du stockage est un problème distinct de la gestion des instances de calcul. Le sous-système PersistentVolume fournit une API pour les utilisateurs et les administrateurs qui extrait les détails de la façon dont le stockage est fourni et de la façon dont il est consommé. Pour ce faire, nous introduisons deux nouvelles ressources API : **PersistentVolume** et **PersistentVolumeClaim**.

Exercice 6.2 : Créer un Persistant Volume (PV)

Un PersistentVolume (PV) est un élément de stockage dans le cluster qui a été provisionné par un administrateur ou provisionné dynamiquement à l'aide de classes de stockage. C'est une ressource dans le cluster, tout comme un nœud est une ressource de cluster. Les PV sont des plugins comme des Volumes, mais ont un cycle de vie indépendant de tout Pod individuel qui utilise le PV. Cet objet API capture les détails de la mise en œuvre du stockage, qu'il s'agisse d'un PVC, NFS, d'iSCSI ou d'un système de stockage spécifique au fournisseur de cloud.

1. créons un fichier YAML pour l'objet avec le type PersistentVolume. Utilisons le nom pvvol-1 pour dire volume1 de type persistant Volume. Seule la syntaxe est vérifiée, un nom ou un répertoire incorrect va générer une erreur, mais un Pod utilisant la ressource ne démarrera pas pour l'instant. Notons que les accessModes n'affectent pas actuellement les données réelles.

controlplane $ vim PVol.yaml

PVol.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pvvol-1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: slow
  hostPath:
    path: /data
```

2. Créons le volume persistant, puis on vérifie bien sa création.

controlplane $ kubectl apply -f PVol.yaml

```
persistentvolume/pvvol-1 created
```

controlplane $ kubectl get pv

```
NAME        CAPACITY    ACCESS MODES    RECLAIM POLICY    STATUS        CLAIM
STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
pvvol-1   1Gi      RWX        Retain        Available        slow        <unset>                13s
```

Exercice 6.3 : Créer un Persistant Volume Claim (PVC)

Avant que les pods puissent profiter du nouveau PV, nous devons créer un Persistant Volume Claim ou l'abréviation c'est PVC.

1. Commençons par déterminer s'il en existe actuellement un pvc de créer.

controlplane $ kubectl get pvc

```
No resources found in default namespace.
```

controlplane $ vim pvc.yaml

pvc.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-one
spec:
  accessModes:
    - ReadWriteMany
  volumeMode: Filesystem
  resources:
    requests:
      storage: 200Mi
  storageClassName: slow
```

2. Créons et vérifions que le nouveau PVC est lié. La taille alloué est de 1Gi, même si 200Mi ont été utilisé . Seul un volume d'une taille inférieur pourrait être utilisé.

controlplane $ kubectl apply -f pvc.yaml

```
persistentvolumeclaim/pvc-one created
```

3. Examinons à nouveau l'état du pv pour déterminer s'il est en cours d'utilisation. Il devrait afficher un statut Lié.

controlplane $ kubectl get pvc

```
NAME     STATUS   VOLUME    CAPACITY   ACCESS MODES   STORAGECLASS
VOLUMEATTRIBUTESCLASS   AGE
pvc-one   Bound    pvvol-1   1Gi        RWX            slow           <unset>                 19s
```

Exercice 6.3 : Créer un Pod utilisant un PVC

Créons à présent un nouveau pod pour utiliser le pvc. Nous allons copier et modifier un fichier yaml d'un pod existant sur le site officiel de Kubernetes . Nous allons ensuite changer le nom du pod ou pas, puis ajoutez une section **volumeMounts** sous le conteneur et une section volumes à la spécification générale ici le nom est « **mypd** » . Le nom utilisé doit être le même aux deux endroits, quel que soit le nom que nous avons utilisé. Le **ClaimName** doit correspondre à un PVC existant. Comme le montre l'exemple suivant :

controlplane $ vim mypod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
      - mountPath: "/var/www/html"
        name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: pvc-one
~
```

controlplane $ kubectl apply -f mypod.yaml

```
pod/mypod created
```

controlplane $ kubectl describe po mypod | grep Volumes: -A3

```
Volumes:
 mypd:
  Type:      PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
  ClaimName:  pvc-one
```

Enfin regardons les détails du pod. On distingue bien qu'il utilise le pvc déjà créer dans la section précédente **pvc-one** qui utilise une taille de 200 Mo parmi les 1Go déjà alloué.

Chapitre 7

Services

7 Laboratoires

Exercice 7.1 : Qu'est-ce qu'un service dans Kubernetes

Les services (également appelés microservices) sont des objets qui déclarent une politique pour **accéder** à un ensemble logique de pods. Des étiquettes (labels) leur sont généralement attribuées pour permettre un accès persistant à une ressource.

Les principaux types de service sont :

• **ClusterIP** par défaut - expose sur une adresse IP interne au cluster. Accessible uniquement au sein du cluster. Comme le montre la figure ci-dessous :

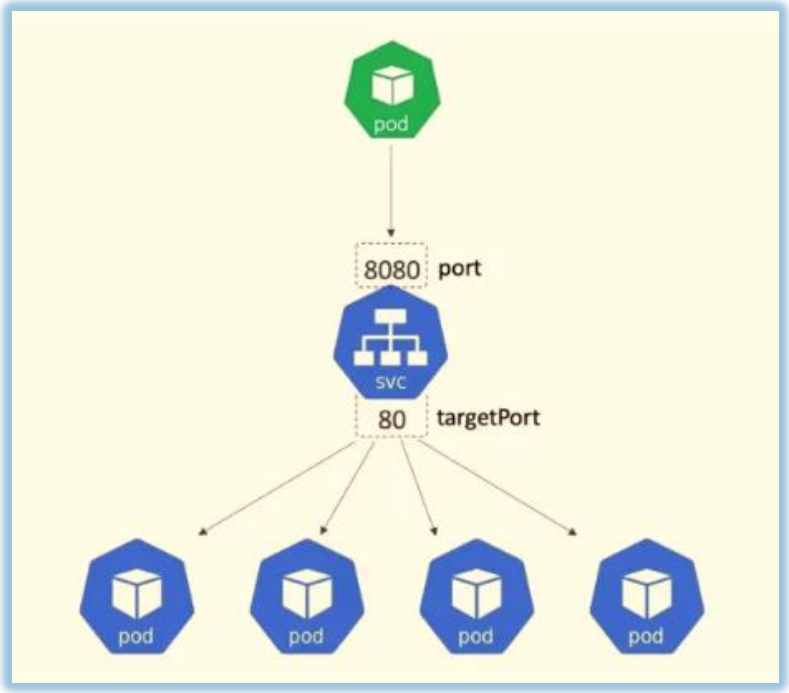

Figure VI : Service ClusterIP

• **NodePort** Expose l'adresse IP du nœud sur un port statique.Un ClusterIP est également automatiquement créé. Comme le montre la figure ci-dessous :

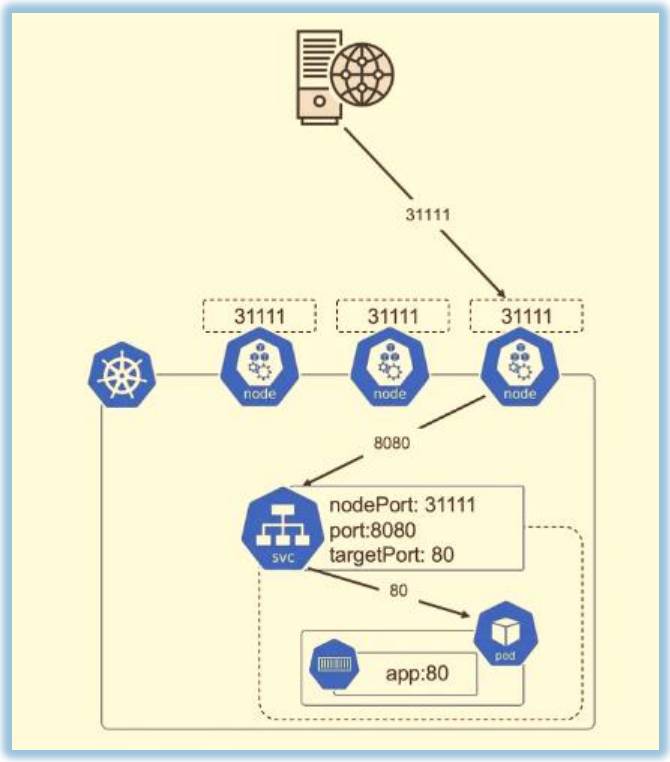

Figure VII : Service NodePort

• **LoadBalancer** expose le service en **externe** à l'aide de l'équilibreur de charge des fournisseurs de cloud. NodePort et ClusterIP créés automatiquement. Comme le montre la figure ci-dessous :

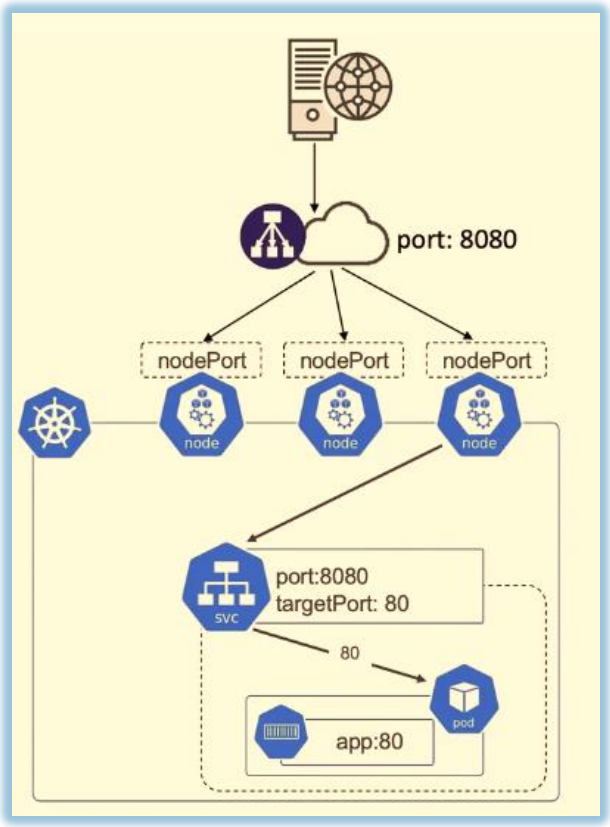

Figure VIII : Service LoadBalancer

Source des images pour service : https://nigelpoulton.com/explained-kubernetes-service-ports/

Exercice 7.2 : Configurer un service ClusterIP

1. Créons un déploiement nginx avec deux replicas à l'aide de kubectl et prenons comme image nginx sur le port 8080. Les étiquettes (labels) dans ce fichier ont été tous configuré avec la clé **system** et la valeur **secondary** .

controlplane $ vim nginx-one.yaml

nginx-one.yaml

```yaml
apiVersion: apps/v2
kind: Deployment
metadata:
  name: nginx-one
  labels:
    system: secondary
spec:
  selector:
    matchLabels:
      system: secondary
  replicas: 2
  template:
    metadata:
      labels:
        system: secondary
    spec:
      containers:
      - image: nginx:1.20.1
        imagePullPolicy: Always
        name: nginx
        ports:
        - containerPort: 8080
          protocol: TCP
      nodeSelector:
        system: secondOne
```

2. Affichons l'état des nouveaux pods. On remarque que l'état des pods ne sont pas en cours d'exécution status « pending ».

controlplane $ kubectl apply -f nginx-one.yaml

```
deployment.apps/nginx-one created
```

controlplane $ kubectl get deployments.apps

```
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-one   0/2          2                0        8s
```

controlplane $ kubectl get po

```
NAME                          READY   STATUS    RESTARTS   AGE
nginx-one-5bdc6ddf4b-8klk8    0/1     Pending   0          15s
nginx-one-5bdc6ddf4b-xpfsg    0/1     Pending   0          15
```

3. La description de l'un des pods nous montre qu'il y a un problème d'affinité ou de selecteur de nœud. On peut résoudre ceci facilement si l'on regarde le code ci-dessus. Dans la partie **nodeSelector** on voit bien que la clé est **system** et la valeur est

secondOne qui est différente de tous les autres valeurs. Donc pour pallier à ce problème on ajoute le label (étiquette) secondOne dans le nœud ou l'on veut exécuté notre deploiement.

controlplane $ kubectl describe pod nginx-one-5bdc6ddf4b-8klk8

```
Name:           nginx-one-5bdc6ddf4b-8klk8
Namespace:      default
Priority:       0
Service Account:  default
Node:           <none>

<output_omitted>

Events:
  Type     Reason            Age   From               Message
  ----     ------            ----  ----               -------
  Warning  FailedScheduling  2m8s  default-scheduler  0/2 nodes are available: 2 node(s) didn't match Pod's node affinity/selector. preemption: 0/2 nodes are available: 2 Preemption is not helpful for scheduling.
```

controlplane $ kubectl get nodes

```
NAME          STATUS   ROLES           AGE   VERSION
controlplane  Ready    control-plane   23d   v1.29.0
node01        Ready    <none>          23d   v1.29.0
```

4. Étiquetez le nœud avec la clé **system** et la valeur **secondOne**. Notons que la valeur est sensible à la casse.

controlplane $ kubectl label node node01 system=secondOne

```
node/node01 labeled
```

controlplane $ kubectl get nodes node01 --show-labels | grep -i secondone

```
node01   Ready   <none>   23d   v1.29.0   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=node01,kubernetes.io/os=linux,system=secondOne
```

5. Après un instant (ça peut varier entre quelques secondes à une ou deux minutes) on retrouve l'état souhaité de nos pods qui est Running.

controlplane $ kubectl get pods

```
NAME                    READY   STATUS    RESTARTS   AGE
nginx-one-5bdc6ddf4b-8klk8   1/1     Running   0          4m31s
nginx-one-5bdc6ddf4b-xpfsg   1/1     Running   0          4m31s
```

6. Rappelons que nous avons exposé le port 8080 dans le fichier YAML. Exposons à present le nouveau déploiement.

controlplane $ kubectl expose deployment nginx-one

```
service/nginx-one exposed
```

controlplane $ kubectl get svc

```
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)      AGE
kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP      23d
nginx-one    ClusterIP   10.100.221.22  <none>        8080/TCP     13s
```

7. Affichons aussi les points de terminaison nouvellement exposés. On voit bien que le port 8080 a été exposé sur chaque pod.

controlplane $ kubectl get ep nginx-one

```
NAME        ENDPOINTS                                   AGE
nginx-one   192.168.1.4:8080,192.168.1.5:8080   29s
```

8. On remarque que la tentative d'accès au Pod sur le port 8080 a échoué. Même si nous avons exposé le port 8080 du conteneur, le L'application à l'intérieur n'a pas été configurée pour écouter sur ce port. Le serveur nginx écoute sur le port 80 par défaut. C'est pour cela la deuxième tentative sur le port 80 avec la commande curl nous a renvoyé la page de bienvenue typique de nginx .

controlplane $ curl 192.168.1.4:8080

```
curl: (7) Failed to connect to 192.168.1.4 port 8080: Connection refused
```

controlplane $ curl 192.168.1.4:80

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

9. Supprimons le déploiement puis Modifions le fichier YAML pour exposer sur le port 80 au lieu du port 8080 et recréez à nouveau le déploiement pour la section suivante.

controlplane $ kubectl delete deploy nginx-one

```
deployment.apps "nginx-one" deleted
```

Exercice 7.3: Configurer un service NodePort

Dans cet exercice nous allons déployer un **NodePort**. Bien que nous puissions accéder à un conteneur depuis le cluster, on préfère utiliser un trafic NodePort vers NAT de l'extérieur du cluster. L'une des raisons de déployer un NodePort à la place d'un LoadBalancer est également une ressource d'équilibrage de charge auprès de fournisseurs de cloud comme Google Cloud et AWS.

1. Créons d'abord un espace de noms (**namespace**). On lui donnera le nom **accounting** (comptable). C'est la ou l'on va héberger nos pods, déploiements, services et autres.

controlplane $ kubectl create ns accounting

```
namespace/accounting created
```

controlplane $ kubectl get ns

```
NAME              STATUS   AGE
accounting        Active   12s
default           Active   23d
kube-node-lease   Active   23d
kube-public       Active   23d
kube-system       Active   23d
local-path-storage   Active   23d
```

2. Vérifions dans le fichier **nginx-one.yaml** qu'on a rajouter un **namespace** avec la valeur **accounting**. Puis on refait pratiquement les mêmes étapes que la section précédente.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-one
  labels:
    system: secondary
  namespace: accounting
```

controlplane $ kubectl create -f nginx-one.yaml

```
deployment.apps/nginx-one created
```

controlplane $ kubectl get po -n accounting

```
NAME                        READY   STATUS    RESTARTS   AGE
nginx-one-8697dd5b94-kvhtq   0/1     Pending   0          8m58s
nginx-one-8697dd5b94-wg2cf   0/1     Pending   0          8m58s
```

controlplane $ kubectl get deployments.apps -n accounting

```
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-one   0/2     2            0           9m11s
```

controlplane $ kubectl label node node01 system=secondOne

```
node/node01 labeled
```

controlplane $ kubectl get deployments.apps -n accounting

```
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-one   2/2     2            2           10m
```

3. Lors d'une étape précédente, nous avons pu afficher la page nginx en utilisant l'adresse IP interne du pod. Exposons maintenant le déploiement en utilisant **--type=NodePort**. Nous lui donnerons également un nom facile à retenir et le placerons dans l'espace de noms **accounting**. Nous pourrions également faire de la translation de port, ce qui pourrait aider à ouvrir des ports dans le pare-feu s'il en existe un.

controlplane $ kubectl -n accounting expose deployment nginx-one --type=NodePort --name=service-lab

```
service/service-lab exposed
```

4. Affichons les détails des services dans l'espace de noms accounting. Nous recherchons le port généré automatiquement. On le voit bien c'est 31099.

controlplane $ kubectl -n accounting describe services

```
Name:                     service-lab
Namespace:                accounting
Labels:                   system=secondary
Annotations:              <none>
Selector:                 system=secondary
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.104.164.111
IPs:                      10.104.164.111
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  31099/TCP
Endpoints:                192.168.1.4:80,192.168.1.5:80
```

5. Localisons le nom d'hôte ou l'adresse IP extérieur du cluster. L'atelier recommande l'utilisation de nœuds auxquels nous accédons via une FloatingIP (adresse IP publique). Or c'est loin d'être le cas dans ce lab, nous nous contenterons d'une adresse IP interne uniquement. Dans notre cas notre master a pour nom controlplane et son adresse IP interne est 172.30.1.2.

controlplane $ kubectl cluster-info

```
Kubernetes control plane is running at https://172.30.1.2:6443
CoreDNS is running at https://172.30.1.2:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

6. Testons l'accès au serveur Web nginx en utilisant l'adresse IP du cluster et le NodePort

controlplane $ curl http://172.30.1.2:31099

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Exercice 7.4: Configurer un service LoadBalancer sur EKS de AWS

1. Créons un cluster sur AWS en se basant sur le service EKS qu'on nommera **clusterK8S**

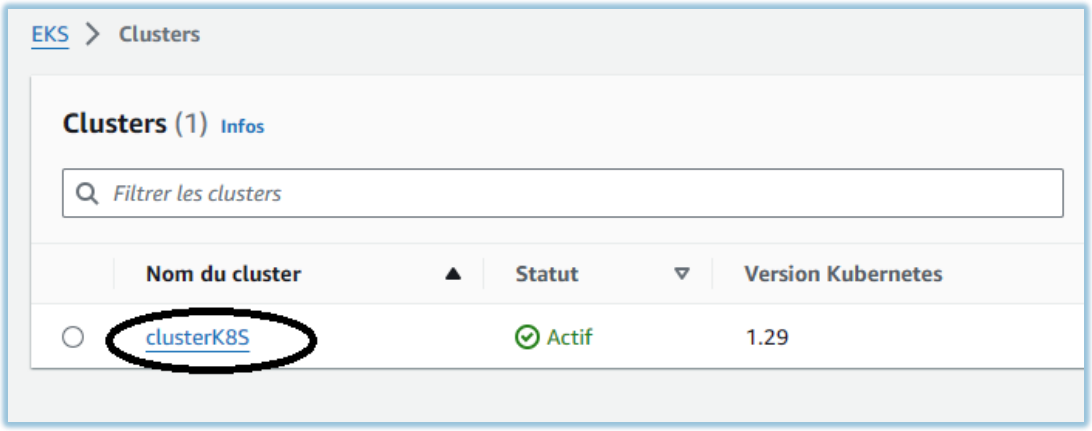

Figure IX : Creation cluster Kubernetes sur AWS

2. La commande ci-dessous met à jour la configuration **kubeconfig** pour permettre l'accès au cluster Kubernetes nommé "**clusterK8S**" situé dans la région EU (Paris) d'Amazon EKS. Cela nous permettra d'exécuter des commandes Kubernetes sur ce cluster à partir de la ligne de commande.

[grass.DESKTOP-KVSHJ9R] ➤ aws eks update-kubeconfig --region eu-west-3 --name clusterK8S

```
Added new context arn:aws:eks:eu-west-3:797573075112:cluster/clusterK8S to
C:\Users\grass\.kube\config
```

3. Voyons combien on a de nœud. Pour le moment j'ai qu'un seul nœud qui va jouer à la fois le rôle de master et le rôle de worker.

[grass.DESKTOP-KVSHJ9R] ➤ kubectl get node

```
NAME                                        STATUS   ROLES    AGE    VERSION
ip-172-31-36-127.eu-west-3.compute.internal   Ready    <none>   5m15s   v1.29.0-eks-5e0fdde
```

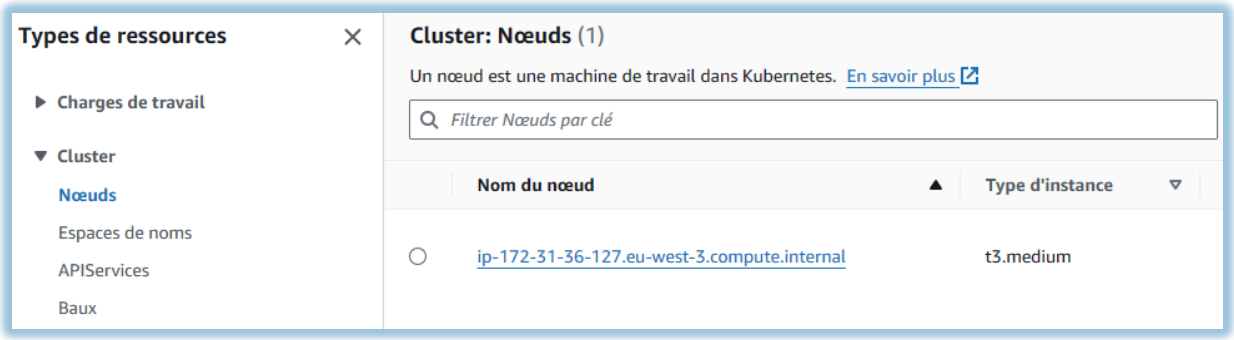

Figure X : Cluster Kubernetes avec un seul nœud

4. Voyons plus de détails sur notre nœud avec l'option **-o wide** et vérifions au même temps qu'on a pas de ressource pour le moment.

[grass.DESKTOP-KVSHJ9R] ➤ kubectl get node -owide

```
NAME                          STATUS   ROLES    AGE    VERSION           INTERNAL-IP-IMAGE
ip-172-31-36-127.eu*          Ready    <none>   5m28s  v1.29.0-eks-5e0fdde   172.31.36.127
EXTERNAL-IP-KERNEL-VERSION                        OS-CONTAINER-RUNTIME
        13.38.118.228          Amazon Linux 2   5.10.210-201.852.amzn2.x86_64   containerd://1.7.11
```

[grass.DESKTOP-KVSHJ9R] ➤ kubectl get pod

```
No resources found in default namespace.
```

[grass.DESKTOP-KVSHJ9R] ➤ vim nginx-pod.yaml

```yaml
nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx-container
    image: nginx:latest
    ports:
    - containerPort: 80
```

[grass.DESKTOP-KVSHJ9R] ➤ kubectl apply -f nginx-pod.yaml

```
Pod/nginx-pod created
```

[grass.DESKTOP-KVSHJ9R] ➤ kubectl get pod

```
NAME       READY   STATUS    RESTARTS   AGE
nginx-pod  1/1     Running        0      8s
```

5. voyons sur le fournisseur de cloud AWS si l'on a bien un pod de créer qui se nomme **nginx-pod**

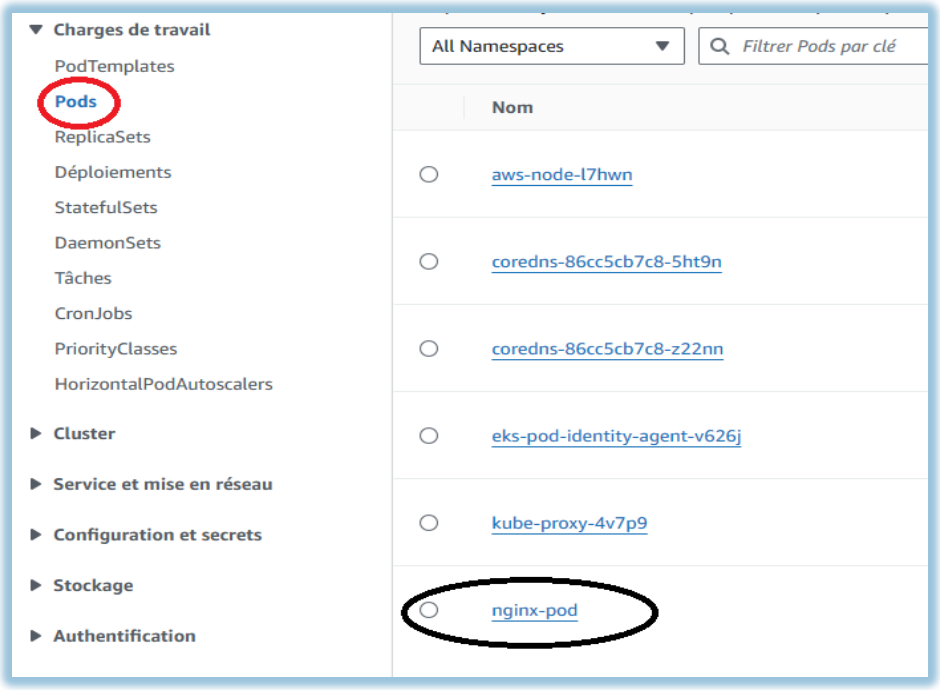

Figure XI : Visualisation des pods sur le cluster Kubernetes EKS de AWS

6. On va tenter de créer un service qui se nomme **nginx-service** avec le type **Loadbalancer**, on remarque qu'il nous rejette parce notre pod n' a pas de label.

[grass.DESKTOP-KVSHJ9R] ➤ kubectl expose pod nginx-pod --name=nginx-service --type=LoadBalancer --port=80 --target-port=80

```
error: couldn't retrieve selectors via --selector flag or introspection: the pod has no labels and cannot be exposed
```

7. Créons un label pour le pod en quesion avec la clé **app** et la valeur **nginx-pod**

[grass.DESKTOP-KVSHJ9R] ➤ kubectl label pod nginx-pod app=nginx-pod

```
pod/nginx-pod labeled
```

8. Retentions de nouveau d'exposé le pod nginx-pod avec le type Loadbalancer. Cette fois ça fonctionne. D'ailleurs on le voit bien dans AWS EKS un peu plus bas

[grass.DESKTOP-KVSHJ9R] ➤ kubectl expose pod nginx-pod --name=nginx-service --type=LoadBalancer --port=80 --target-port=80

```
service/nginx-service exposed
```

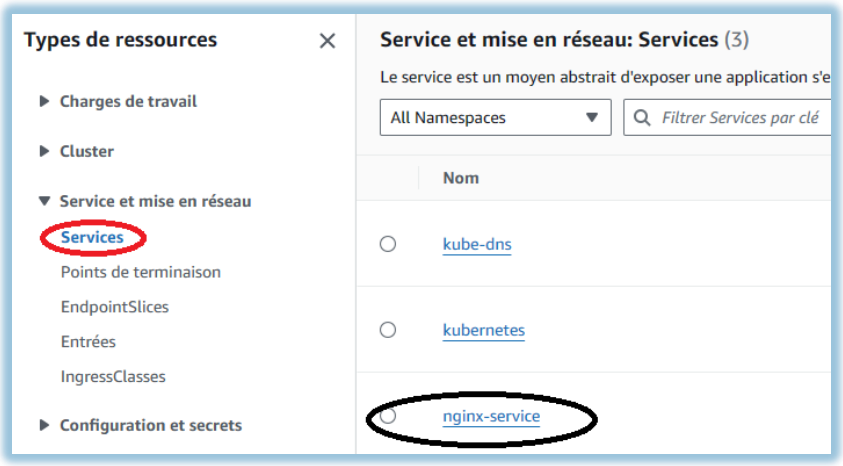

Figure XII: Visualisation des services dans EKS de AWS

9. Tout cette acharnement pour en arriver là et de voir dans la partie **EXTERNAL-IP** soit un **URL** pour accéder depuis l'extérieur à notre serveur nginx ou bien en utilisant une adresse IP publique. Notons au passage que j'ai activé le port 80 dans la partie sécurity Group sur AWS pour ne pas être bloqué au niveau du Firewall.

[grass.DESKTOP-KVSHJ9R] ➤ kubectl get svc

```
NAME          TYPE          CLUSTER-IP     EXTERNAL-IP                                                    PORT(S)    AGE
kubernetes    ClusterIP     10.100.0.1     <none>                                                         443/TCP    46m
nginx-service LoadBalancer  10.100.209.65  aa6af6cec4e8a4704a72a0ff5fc0be28-678294017.eu-west-3.elb.amazonaws.com  80:31118/TCP  5s
```

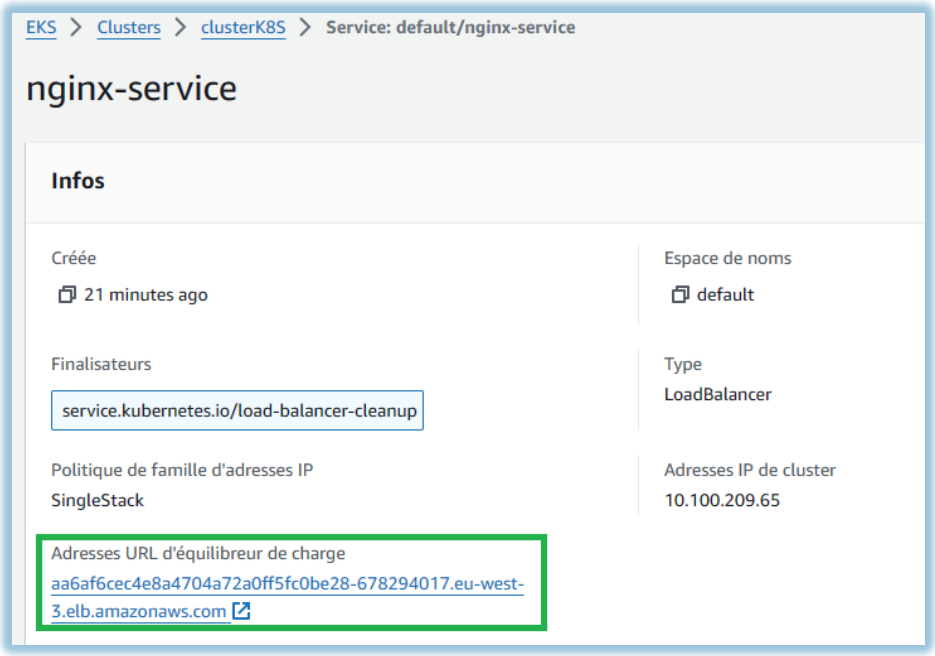

Figure XIII: L'URL pour accéder à la page web dans un navigateur

[grass.DESKTOP-KVSHJ9R] ➤ kubectl get endpoints

```
NAME          ENDPOINTS                    AGE
kubernetes    172.31.18.100:443,172.31.9.138:443   47m
nginx-service 172.31.47.187:80             56s
```

10. Accès à notre serveur web avec la commande **curl** depuis la ligne de commande pour s'assurer qu'il n'y a pas d'empêchement.

[grass.DESKTOP-KVSHJ9R] ➤ curl aa6af6cec4e8a4704a72a0ff5fc0be28-678294017.eu-west-3.elb.amazonaws.com

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
```

11. Accès avec le même URL sur un browser depuis l'extérieur et de n'importe où dans le monde

Figure XIV: Affichage de la page NGINX avec L'URL fourni dans un navigateur

Chapitre 8

Scheduling (Planification)

8 Laboratoires

Exercice 8.1: Attribuer des pods à l'aide des labels (étiquettes)

On souhaite dans cette section de déterminer un pod utilisera quel(s) nœud(s). Par exemple, on peut avoir des exigences matérielles particulières à respecter pour la charge de travail. Dans ce cas on attribue les pods VIP aux nouveaux matériels plus rapide et plus performants. Tandis que les autres pods seront attribuer au reste du parc informatique un peu plus ancien. Dans cet exercice, nous utiliserons des ***labels*** (étiquette) pour programmer des pods sur un nœud particulier. Ensuite, nous explorerons les ***taints*** (taches) pour avoir un déploiement plus flexible dans un environnement plus large.

1. Donnons une liste des nœuds. Ils doivent être à l'état prêts et sans labels ni taints ajoutées.

controlplane $ kubectl get nodes

```
NAME          STATUS   ROLES          AGE   VERSION
controlplane   Ready    control-plane   26d   v1.29.0
node01         Ready    <none>          26d   v1.29.0
```

2. Affichons les labels et les taints actuelles pour nos deux nœuds **controlplane** et **node01**.

controlplane $ kubectl describe nodes |grep -A5 -i label

```
Labels:          beta.kubernetes.io/arch=amd64
                 beta.kubernetes.io/os=linux
                 kubernetes.io/arch=amd64
                 kubernetes.io/hostname=controlplane
                 kubernetes.io/os=linux
                 node-role.kubernetes.io/control-plane=
--
```

```
Labels:         beta.kubernetes.io/arch=amd64
                beta.kubernetes.io/os=linux
                kubernetes.io/arch=amd64
                kubernetes.io/hostname=node01  kubernetes.io/os=linux
```

controlplane $ kubectl describe nodes |grep -i taint

```
Taints:         <none>
Taints:         <none>
```

3. On va à présent compter le nombre de conteneurs exécutés sur les deux nœuds. Il y a environ 10 conteneurs exécutés sur le controlplane dans l'exemple suivant, et six sur le node01. La commande **wc** nous donne le nombre de ligne avec l'option **-l** . nous pouvons en avoir plus ou moins, en fonction de nos ressources (pods) . Avec cette commande on peut prendre note du nombre de conteneurs, puis on conclut que les chiffres changent au fur et à mesure de la planification (scheduling). Le changement entre les nœuds est une information importante, et non pas un nombre aléatoire. Ici on utilise **cri-o**, donc pour afficher les conteneurs en utilisant **crictl ps**.

controlplane $ kubectl get deployments --all-namespaces

```
NAMESPACE         NAME                      READY   UP-TO-DATE   AVAILABLE   AGE
kube-system       calico-kube-controllers   1/1     1            1           26d
kube-system       coredns                   2/2     2            2           26d
local-path-storage   local-path-provisioner   1/1     1            1           26d
```

controlplane $ sudo crictl ps | wc -l
10
controlplane $ ssh node01
Last login: Sun Nov 13 17:27:09 2022 from 10.48.0.33
node01 $ sudo crictl ps | wc -l
6
node01 $ exit

```
logout
Connection to node01 closed.
```

4. On désire par exemple attribuer au nœud controlplane le matériel VIP et au nœud node01 le reste du matériels. Donc fixons les labels :

controlplane $ kubectl label nodes controlplane status=vip

```
node/controlplane labeled
```

controlplane $ kubectl label nodes node01 status=other

```
node/node01 labeled
```

5. Regardons un peu nos paramètres. Nous trouverons des labels intégrées telles que le nom d'hôte, le système d'exploitation et le type d'architecture. Le format ci-dessous apparaît sur plusieurs lignes pour plus de lisibilité.

controlplane $ kubectl get nodes --show-labels

```
NAME          STATUS   ROLES          AGE   VERSION   LABELS
controlplane   Ready    control-plane   26d   v1.29.0
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes
.io/hostname=controlplane,kubernetes.io/os=linux,node-role.kubernetes.io/control-
plane=,node.kubernetes.io/exclude-from-external-load-balancers=,status=vip
node01        Ready    <none>         26d   v1.29.0
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes
.io/hostname=node01,kubernetes.io/os=linux,status=other
```

6. Créons un fichier **vip.yaml** pour générer quatre conteneurs qui dorment tout le temps. Notons qu'au départ on a donné le **status vip** au **nodeSelector.**

controlplane $ vi vip.yaml

vip.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: vip
spec:
  containers:
  - name: vip1
    image: busybox
    args:
    - sleep
    - "1000000"
  - name: vip2
    image: busybox
    args:
    - sleep
    - "1000000"
  - name: vip3
    image: busybox
    args:
    - sleep
    - "1000000"
  - name: vip4
    image: busybox
    args:
    - sleep
    - "1000000"
  nodeSelector:
    status: vip
```

controlplane $ kubectl create -f vip.yaml

```
pod/vip created
```

7. Déployons le nouveau pod. Vérifions que les conteneurs ont été créés sur le nœud **controlpane**. Cela peut prendre quelques secondes pour que tous les conteneurs apparaissent. À partir de maintenant, on utilisera la commande **crictl** pour voir l'état des conteneurs sous-jacents.

controlplane $ kubectl get po -owide

```
NAME   READY   STATUS    RESTARTS   AGE   IP          NODE          NOMINATED NODE   READINESS
GATES
vip    4/4     Running   0          11s   192.168.0.4   controlplane   <none>           <none>
```

8. Dans ce qui suit on va créer un fichier **/etc/crictl.yaml.** Son rôle est de rendre accessible le point d'extrémité de runtime via le socket Unix **containerd.sock** . Cela signifie qu'il existe une communication entre le moteur de conteneurisation et **containerd** (un moteur de conteneur open source), qui utilise un socket Unix comme mécanisme de communication pour effectuer des opérations de gestion des

conteneurs, telles que le démarrage, l'arrêt et aussi la vérification d'état des conteneurs avec la commande **ps** .

```
controlplane $ sudo cat <<EOF | sudo tee /etc/crictl.yaml
> runtime-endpoint: unix:///run/containerd/containerd.sock
> timeout: 10
> debug: false
> EOF
```

9. La commande **crictl ps**, donne une liste des conteneurs en cours d'exécution sur le système, similaires à la commande "**docker ps".** Cela peut inclure des informations telles que l'ID du conteneur, le nom du conteneur, l'ID du pod auquel il appartient, l'ID de l'image utilisée pour le conteneur, l'état du conteneur, etc.

```
controlplane $ sudo crictl ps |wc -l
14
controlplane $ kubectl delete pod vip
```

```
pod "vip" deleted
```

10. éditons le fichier **vip.yaml** et mettons des commentaires sur les lignes du nodeSelector

```
controlplane $ vim vip.yaml
```

```
....
#  nodeSelector:
 #    status: vip
```

11. Créons à nouveau le pod. Les conteneurs peuvent désormais apparaître sur l'un ou l'autre des nœuds. Mais ici l'avantage va toujours vers le second nœud puisque le master à la base n'est pas conçu d'héberger des pods.

```
controlplane $ kubectl create -f vip.yaml
```

```
pod/vip created
```

```
controlplane $ kubectl get po -owide
```

```
NAME   READY   STATUS    RESTARTS   AGE   IP          NODE     NOMINATED NODE   READINESS GATES
vip    4/4     Running   0          3s    192.168.1.5 node01   <none>           <none>
```

12. Créons un autre fichier pour les autres utilisateurs. Changeons aussi tous les mots « vip » dans le fichier par le mot « other » avec la commande **sed** . Enfin il faut décommenter les lignes du nodeSelector

```
controlplane $ cp vip.yaml other.yaml
controlplane $ sed -i s/vip/other/g other.yaml
controlplane $ vim other.yaml
```

other.yaml

```yaml
....
  nodeSelector:
    status: other
```

13. Créons les conteneurs other puis déterminons sur quel nœud vont-ils s'exécuté. Le résultat obtenu est logique vu que on a mis la valeur **other** dans **status** dans le nodeSelector

```
controlplane $ kubectl create -f other.yaml
```

```
pod/other created
```

```
controlplane $ kubectl get po -owide
```

```
NAME    READY   STATUS    RESTARTS   AGE    IP          NODE        NOMINATED NODE   READINESS
GATES
other   4/4     Running   0          6s     192.168.1.4 node01      <none>           <none>
vip     4/4     Running   0          4m39s  192.168.0.4 controlplane <none>         <none>
```

14. En guise de conclusion on remarque que la commande **sudo crictl ps |wc -l** dans son ensemble compte le nombre de conteneurs en cours d'exécution sur les deux noeuds. Et varient selon le type de planification désirée. C'est pour cette raison les nombres afficher sur les deux nœuds sont parfois différents.

```
controlplane $  sudo crictl ps |wc -l
10
controlplane $ ssh node01 "sudo crictl ps |wc -l"
14
controlplane $ kubectl delete pods vip other
```

```
pod "vip" deleted
pod "other" deleted
```

Exercice 8.2 : Utiliser les Taints pour contrôler le dépoloiement des pods

Utilisez des taints pour gérer l'endroit où les pods sont déployés ou autorisés à s'exécuter. En plus d'attribuer un pod à un groupe de nœuds, avec les taints on peut également limiter l'utilisation sur un nœud. Si on veut évacuer complètement les pods sur un nœud quelconque on peut utiliser les taints car c'est un moyen d'y parvenir aussi. Il est important de savoir que le nœud master a déjà une taint NoSchedule dès le départ. Nous travaillerons dans cette section avec trois taints pour limiter ou supprimer les pods en cours d'exécution

1. Créez un déploiement avec huit conteneurs nginx.

controlplane $ vim taint.yaml

taint.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taint-deployment
spec:
  replicas: 8
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name:  nginx
        image: nginx:1.20.1
        ports:
        - containerPort: 80
```

controlplane $ kubectl apply -f taint.yaml

```
deployment.apps/taint-deployment created
```

2. Déterminons où les conteneurs s'exécutent. Dans l'exemple suivant, quatre ont été déployés sur le nœud controlplane et quatre sur le nœud node01. Les chiffres peuvent être différents, le nombre réel n'est pas important, nous suivons l'évolution des chiffres au niveau des conteneurs avec la commande **crictl**.

controlplane $ kubectl get po -owide

```
NAME                                    READY   STATUS    RESTARTS   AGE   IP            NODE
taint-deployment-56597bb4b6-478xb       1/1     Running   0          13s   192.168.0.4   controlplane
taint-deployment-56597bb4b6-9hj98       1/1     Running   0          12s   192.168.1.6   node01
taint-deployment-56597bb4b6-b699w       1/1     Running   0          12s   192.168.0.7   controlplane
taint-deployment-56597bb4b6-gkl9s       1/1     Running   0          12s   192.168.1.7   node01
taint-deployment-56597bb4b6-lxqk9       1/1     Running   0          13s   192.168.1.4   node01
taint-deployment-56597bb4b6-ml52v       1/1     Running   0          12s   192.168.0.6   controlplane
taint-deployment-56597bb4b6-tlt2n       1/1     Running   0          12s   192.168.0.5   controlplane
taint-deployment-56597bb4b6-xfscz       1/1     Running   0          13s   192.168.1.5   node01
```

controlplane $ sudo crictl ps |wc -l

14

controlplane $ ssh node01 "sudo crictl ps |wc -l"

10

3. Supprimons le déploiement et Vérifions par exemple sur le **controlplane** que quatre conteneurs ont disparu. On passe à dix au lieu de quatorze.

controlplane $ kubectl delete deployment taint-deployment

```
deployment.apps "taint-deployment" deleted
```

controlplane $ sudo crictl ps |wc -l

10

4. Nous allons maintenant utiliser une teinte (taint) pour affecter le déploiement de nouveaux conteneurs. Il existe trois taints : **NoSchedule**, **PreferNoSchedule** et **NoExecute**. Les taints liées aux plannings seront utilisées pour déterminer les conteneurs nouvellement déployés, mais n'affecteront pas les conteneurs en cours d'exécution. L'utilisation de NoExecute entraînera le déplacement des conteneurs en cours d'exécution. Appliquons un taint sur le nœud secondaire node01 et vérifions qu'il est teinté, puis créons à nouveau le déploiement. Nous utiliserons la clé de **bubba** pour montrer que le nom de la clé n'est qu'une chaîne qu'un administrateur peut utiliser pour suivre les pods.

controlplane $ kubectl taint nodes node01 bubba=value:**PreferNoSchedule**

```
node/node01 tainted
```

controlplane $ kubectl describe node |grep Taint

```
Taints:             <none>
Taints:             bubba=value:PreferNoSchedule
```

controlplane $ kubectl apply -f taint.yaml

```
deployment.apps/taint-deployment created
```

5. Localisons un peu où les conteneurs sont exécutés. Nous pouvons voir qu'il y a plus de conteneurs sur le controlplane, ce qui est normal parce que la plupart des conteneurs de maintenance se trouvent sur le master, mais il y en a eu quand même sur le nœud secondaire node01. Enfin Supprimons le déploiement lorsque on aurait rassemblé les chiffres.

controlplane $ sudo crictl ps |wc -l
18
controlplane $ ssh node01 "sudo crictl ps |wc -l"
6

controlplane $ kubectl delete deployment taint-deployment

```
deployment.apps "taint-deployment" deleted
```

6. Supprimons le taint sur le node01 puis vérifions qu'elle a été supprimée. Pour supprimer le taint la clé doit être utilisée avec un signe moins ajouté à la fin.

controlplane $ kubectl taint nodes node01 bubba-

```
node/node01 untainted
```

controlplane $ kubectl describe node |grep Taint

```
Taints:         <none>
Taints:         <none>
```

7. Cette fois, utilisons la teinte **NoSchedule**, puis on va créer à nouveau le déploiement. Le nœud node01 **ne doit pas** avoir de nouveaux conteneurs, puisque on vient d'interdire d'y mettre quoique ce soit la dessus. Ça pourrait être intéressant pour un administrateur afin de maintenir ce nœud par exemple.

controlplane $ kubectl taint nodes node01 bubba=value:**NoSchedule**

```
node/node01 tainted
```

controlplane $ kubectl apply -f taint.yaml

```
deployment.apps/taint-deployment created
```

```
controlplane $ sudo crictl ps |wc -l
18
controlplane $ ssh node01 "sudo crictl ps |wc -l"
6
```

8. Supprimons la teinte du node01 et supprimons aussi le déploiement. Lorsque aurons déterminé et que tous les conteneurs sont terminés aussi, créons le déploiement à nouveau mais cette fois sans aucune teinte sur les deux nœuds, On remarque que les conteneurs sont plus ou moins répartis sur les deux nœuds.

```
controlplane $  kubectl delete deployment taint-deployment
```

```
deployment.apps "taint-deployment" deleted
```

```
controlplane $ kubectl taint nodes node01 bubba-
```

```
node/node01 untainted
```

```
controlplane $  kubectl apply -f taint.yaml
```

```
deployment.apps/taint-deployment created
```

```
controlplane $  sudo crictl ps |wc -l
14
controlplane $ ssh node01 "sudo crictl ps |wc -l"
10
```

9. Utilisons encore une fois **NoExecute** pour corrompre le nœud node01. Attendons une petite minute puis déterminons si les conteneurs ont **bougé**. L'arrêt des conteneurs DNS peut prendre un certain temps. Certains conteneurs resteront sur le nœud de node01 pour poursuivre la communication depuis le cluster.

```
controlplane $ kubectl taint nodes node01 bubba=value:NoExecute
```

```
node/node01 tainted
```

```
controlplane $  sudo crictl ps |wc -l
20
controlplane $ ssh node01 "sudo crictl ps |wc -l"
4
```

10. Retirez la teinte du node01. Vérifions que tous les conteneurs ne sont pas revenus à leur emplacement précédent.

```
controlplane $  kubectl taint nodes node01 bubba-
```

```
node/node01 untainted
```

```
controlplane $  sudo crictl ps |wc -l
20
controlplane $ ssh node01 "sudo crictl ps |wc -l"
4
controlplane $  kubectl delete deployment taint-deployment
```

```
deployment.apps "taint-deployment" deleted
```

Enfin Supprimons tous déploiements pour libérer des ressources.

Chapitre 9

Journalisation et Débogage (Logging and Troubleshooting)

9 Laboratoires

Exercice 9.1 : Les emplacements des fichiers journaux

Du point de vue d'un nœud, on a la commande **journalctl** entre autre pour afficher les journaux. Nous allons vous monter les emplacements des fichiers journaux, puis d'autres commandes pour afficher les journaux d'un conteneur. Il existe d'autres options de journalisation, comme l'utilisation d'un conteneur **sidecar** dédié au chargement des logs d'un autre conteneur dans un pod. La journalisation du cluster entier n'est pas encore disponible avec Kubernetes. Des logiciels externes sont généralement utilisés, tels que **Fluentd**, qui fait partie de http://fluentd.org/, qui est un autre projet, comme Kubernetes.

1. Jetons un coup d'œil rapide aux fichiers journaux. Si on utilise un cluster Kubernetes basé sur **systemd**. Il faut regarder avant tout les journaux du service **kubelet** du nœud secondaire ici (node01). Chaque nœud aura un contenu différent car celui-ci est spécifique au nœud.

controlplane $ kubectl get node

```
NAME          STATUS   ROLES          AGE   VERSION
controlplane  Ready    control-plane  28d   v1.29.0
node01        Ready    <none>         28d   v1.29.0
```

controlplane $ ssh node01

node01 $ service kubelet status

```
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; vendor preset: enabled)
  Drop-In: /usr/lib/systemd/system/kubelet.service.d
           └─10-kubeadm.conf
   Active: active (running) since Mon 2024-04-01 13:47:13 UTC; 5min ago
```

controlplane $ journalctl -u kubelet | tail -n 3

```
Mar 31 12:18:40 controlplane kubelet[1687]: I0331 12:18:40.117267    1687 scope.go:117] "RemoveContainer" containerID="354ec743eeb13740017eec627037948f22657c4a6ca10255251f146e1710e4e9"
Mar 31 12:18:40 controlplane kubelet[1687]: I0331 12:18:40.121654    1687 kuberuntime_container_linux.go:167] "No swap cgroup controller present" swapBehavior="" pod="kube-system/canal-j6pwg" containerName="calico-node"
Mar 31 12:18:40 controlplane kubelet[1687]: I0331 12:18:40.392430    1687 kuberuntime_container_linux.go:167] "No swap cgroup controller present" swapBehavior="" pod="kube-system/canal-j6pwg" containerName="kube-flannel"
```

2. Les principaux processus Kubernetes s'exécutent désormais dans des conteneurs. On peut les visualiser du point de vue du **conteneur** ou au niveau du **pod**. Utilisons la commande **find** pour localiser le journal **kube-apiserver**. Regardons le format de sortie

controlplane $ sudo find / -name "*apiserver*log"

```
/var/log/containers/kube-apiserver-controlplane_kube-system_kube-apiserver-58b449d968ef78cbc61486ff906d5ea6805b039cb822bef6e0370e62e1c266d6.log

/var/log/containers/kube-apiserver-controlplane_kube-system_kube-apiserver-8b4769efe197e45a10f12d9540898c3a38a4d403ed5bd1e85fec2e37a70d42fb.log
```

3. Jetons un œil à ce fichier de log et voyons si l'on peut détecter des erreurs

controlplane $ sudo cat /var/log/containers/kube-apiserver-controlplane_kube-system_kube-apiserver-8b4769efe197e45a10f12d9540898c3a38a4d403ed5bd1e85fec2e37a70d42fb.log

```
2024-03-31T12:17:42.78222945Z stderr F I0331 12:17:42.772036       1 options.go:222] external host was not specified, using 172.30.1.2
2024-03-31T12:17:42.818156919Z stderr F I0331 12:17:42.800367       1 server.go:148] Version: v1.29.0
<output_omitted>
```

4. Voici une liste de fichiers journaux comme **coredns**, **kube-proxy** et bien d'autres agents de cluster ou l'on peut chercher la cause de la panne et déboguer par la suite.

controlplane $ cd /var/log/pods
controlplane $ ls

```
kube-system_calico-kube-controllers-9d57d8f49-jvt8w_28538fd3-e350-4827-9220-39e7f2db678b
kube-system_canal-j6pwg_803d7502-bf17-4703-b9c1-630a0388aeae
kube-system_etcd-controlplane_a9fb06c3a332674e3dd80789db1e82ef
kube-system_kube-apiserver-controlplane_6f02fb1b149af288ee0923e3d5997c5c
kube-system_kube-controller-manager-controlplane_935b51c776c9bcac8dbf3a7d870b331f
kube-system_kube-proxy-85drq_02f10dda-915b-47ac-a221-0415ce1a4321
kube-system_kube-scheduler-controlplane_b89e0a38d2f630290f39ad72d802321c
local-path-storage_local-path-provisioner-5d854bc5c4-h55kw_9d98b551-e51b-4716-8873-
1a6484a25a32
```

controlplane $ cd kube-system_kube-apiserver-controlplane_6f02fb1b149af288ee0923e3d5997c5c
controlplane $ ls
kube-apiserver
controlplane $ cd kube-apiserver/
controlplane $ ls
1.log 2.log
controlplane $ cat 2.log

```
2024-04-01T13:47:19.229132302Z stderr F I0401 13:47:19.215180       1 options.go:222] external
host was not specified, using 172.30.1.2
2024-04-01T13:47:19.229196801Z stderr F I0401 13:47:19.217249       1 server.go:148] Version:
v1.29.0
<output_omitted>
```

controlplane $ tail -f 2.log

```
<output_omitted>
2024-04-01T13:48:01.415433282Z stderr F I0401 13:48:01.415281       1 controller.go:624] quota
admission added evaluator for: endpoints
2024-04-01T13:48:01.42982044Z stderr F I0401 13:48:01.429715       1 controller.go:624] quota
admission added evaluator for: endpointslices.discovery.k8s.io
```

Pour plus d'information visiter les deux sites ci-dessous :

https://kubernetes.io/docs/tasks/debug-application-cluster/debug-service/

https://kubernetes.io/docs/tasks/debug-application-cluster/determine-reason-pod-failure

Exercice 9.2 : Afficher des résultats des journaux

La sortie standard du conteneur peut être vue via la commande **kubectl logs**. S'il n'y a pas de sortie standard, on ne verra rien. De plus, les logs seront détruites si les conteneurs sont détruites.

1. Affichons tous les pods actuels dans le cluster et dans tous les espaces de noms.

controlplane $ kubectl get po --all-namespaces

```
NAMESPACE      NAME                                      READY   STATUS    RESTARTS      AGE
kube-system    calico-kube-controllers-9d57d8f49-jvt8w   1/1     Running   3 (24m ago)   27d
....
kube-system    coredns-86b698fbb6-8q542                  1/1     Running   1 (24m ago)   27d
kube-system    coredns-86b698fbb6-hqpmj                  1/1     Running   1 (24m ago)   27d
kube-system    etcd-controlplane                         1/1     Running   2 (24m ago)   27d
kube-system    kube-apiserver-controlplane               1/1     Running   2 (24m ago)   27d
kube-system    kube-controller-manager-controlplane      1/1     Running   2 (24m ago)   27d
kube-system    kube-proxy-85drq                          1/1     Running   2 (24m ago)   27d
kube-system    kube-proxy-lhxdd                          1/1     Running   1 (24m ago)   27d
kube-system    kube-scheduler-controlplane               1/1     Running   2 (24m ago)   27d
.....
```

2. Affichons les journaux associés à divers pods en se basant à la commande **kubectl logs.** Généralement le contenu d'un fichier de logs est assez long, pour filtrer ce que l'on cherche on peut utiliser la commande **tail** ou **head,** comme le montre l'exemple ci-dessous :

controlplane $ kubectl -n kube-system logs kube-apiserver-controlplane | tail -n 10

```
I0331 12:18:18.244262       1 controller.go:624] quota admission added evaluator for: endpoints
I0331 12:18:18.253349       1 controller.go:624] quota admission added evaluator for:
endpointslices.discovery.k8s.io
E0331 12:18:21.115558       1 authentication.go:73] "Unable to authenticate the request"
err="[invalid bearer token, service account token has expired]"
E0331 12:18:21.198663       1 authentication.go:73] "Unable to authenticate the request"
err="[invalid bearer token, service account token has expired]"
<output_omitted>
```

Exercice 9.3 : Ajout d'outils de surveillance et de métriques

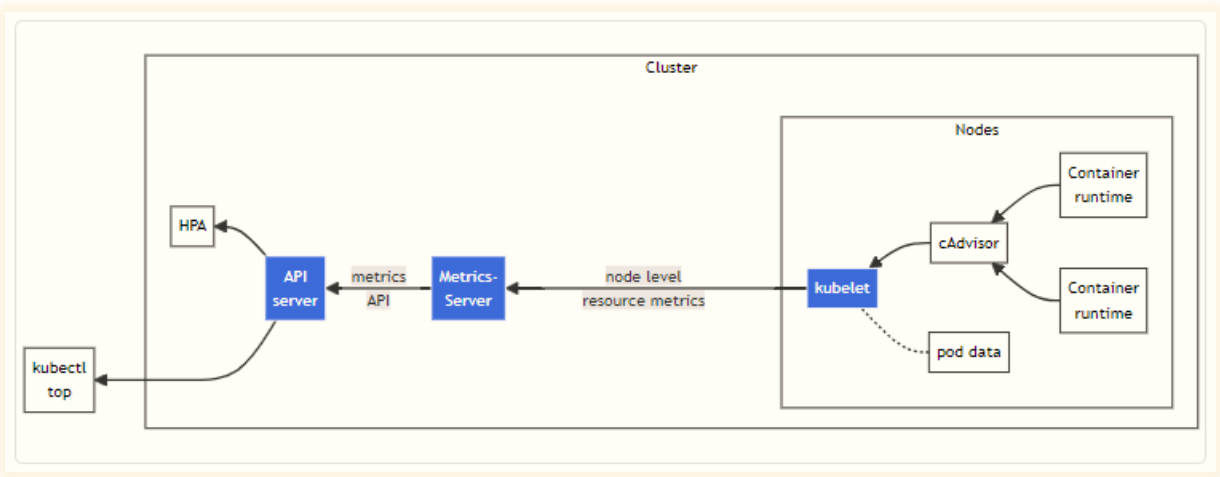

Figure XV: L'architecture du pipeline des métriques de ressources

Configurer les métriques

1. Voyons d'abord si on peut récolter quelques métriques sur nos nœuds au moyen de la commande **kubectl top**

controlplane $ kubectl get node

```
NAME          STATUS   ROLES          AGE    VERSION
controlplane  Ready    control-plane  29d    v1.29.0
node01        Ready    <none>         29d    v1.29.0
```

controlplane $ kubectl top node controlplane

```
error: Metrics API not available
```

controlplane $ kubectl top node node01

```
error: Metrics API not available
```

2. On va chercher sur github le projet de kubernetes nommé **metrics-server** qui va nous permettre d'afficher quelques métriques sur les nœuds.

controlplane $ git clone https://github.com/kubernetes-incubator/metrics-server.git

```
Cloning into 'metrics-server'...
remote: Enumerating objects: 16512, done.
remote: Counting objects: 100% (5409/5409), done.
remote: Compressing objects: 100% (1489/1489), done.
remote: Total 16512 (delta 4275), reused 3958 (delta 3919), pack-reused 11103
Receiving objects: 100% (16512/16512), 14.21 MiB | 20.55 MiB/s, done.
Resolving deltas: 100% (8982/8982), done.
```

3. Créons les objets nécessaires. Il faut noter que de nouvelles versions sont publiées au fur et à mesure, des modifications peuvent être apportées au processus et aux objets créés. Utilisons le **composants.yaml** pour créer les objets

controlplane $ kubectl create -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

```
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
```

4. Si l'on affiche les objets actuels, qui sont créés dans l'espace de noms **kube-system**. Tous devraient afficher un statut En cours d'exécution, sauf le pod du serveur de métriques n'est pas prêt. Pour ce faire il faut autorisez le déploiement à exécuter un TLS non sécurisé et le pod commencera à accepter le trafic.

controlplane $ kubectl -n kube-system get pods

```
NAME                                READY   STATUS    RESTARTS   AGE

<output_omitted>
metrics-server-6d94bc8694-82kf9     0/1     Running   0          110s
```

5. Modifions le déploiement du serveur de métriques pour autoriser TLS non sécurisé. Le certificat par défaut est x509 auto-signé et n'est pas approuvé par défaut. Parfois il faut effacer l'ancien pod pour laisser à la nouvelle configuration de lancer une nouvelle pod.

controlplane $ kubectl -n kube-system edit deployment metrics-server

```yaml
....
    spec:
      containers:
      - args:
        - --cert-dir=/tmp
        - --secure-port=4443
        - --kubelet-insecure-tls  #<-- Ajouter cette ligne
        - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
        image: k8s.gcr.io/metrics-server/metrics-server:v0.3.7
....
```

controlplane $ kubectl -n kube-system get pods

```
NAME                                READY   STATUS    RESTARTS    AGE

<output_omitted>
metrics-server-bcffdb84f-mjt64      1/1     Running   0           60s
```

6. Testons encore une fois le fonctionnement des métriques des nœuds. Et ca marche cette fois, on obtient le résultat voulu.

controlplane $ kubectl top node controlplane

```
NAME         CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
controlplane  147m        14%     1207Mi          64%
```

controlplane $ kubectl top node node01

```
NAME    CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
node01   47m          4%      830Mi           44%
```

7. Voyons en détails le fonctionnement des métriques des pods et des nœuds. On a fait un **kubectl describe** qui va nous donner quelques détails à propos des métriques des nœuds et des pods. Cela peut prendre environ une minute pour que les métriques soient remplies et ne renvoient pas d'erreur.

controlplane $ kubectl describe node controlplane

<output_omitted>

```
  Hostname:    controlplane
Capacity:
  cpu:                1
  ephemeral-storage:  20134592Ki
  hugepages-2Mi:      0
  memory:             2030940Ki
  pods:               110
Allocatable:
  cpu:                1
  ephemeral-storage:  19586931083
  hugepages-2Mi:      0
  memory:             1928540Ki
  pods:               110
```

<output_omitted>

```
  Namespace                   Name                                       CPU Requests  CPU Limits  Memory Requests  Memory Limits  Age
  ---------                   ----                                       ------------  ----------  ---------------  -------------  ---
  kube-system                 calico-kube-controllers-9d57d8f49-jvt8w    0 (0%)        0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 canal-j6pwg                                25m (2%)      0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 etcd-controlplane                          25m (2%)      0 (0%)      100Mi (5%)       0 (0%)         29d
  kube-system                 kube-apiserver-controlplane                50m (5%)      0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 kube-controller-manager-controlplane       25m (2%)      0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 kube-proxy-85drq                           0 (0%)        0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 kube-scheduler-controlplane                25m (2%)      0 (0%)      0 (0%)           0 (0%)         29d
  local-path-storage          local-path-provisioner-5d854bc5c4-h55kw    0 (0%)        0 (0%)      0 (0%)           0 (0%)         29d
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests    Limits
  --------           --------    ------
  cpu                150m (15%)  0 (0%)
  memory             100Mi (5%)  0 (0%)
  ephemeral-storage  0 (0%)      0 (0%)
  hugepages-2Mi      0 (0%)      0 (0%)
```

controlplane $ kubectl describe node node01

<output_omitted>

```
  Hostname:    node01
Capacity:
  cpu:                1
  ephemeral-storage:  20134592Ki
  hugepages-2Mi:      0
  memory:             2030940Ki
  pods:               110
Allocatable:
  cpu:                1
  ephemeral-storage:  19586931083
  hugepages-2Mi:      0
  memory:             1928540Ki
  pods:               110
```

<output_omitted>

```
  Namespace                   Name                              CPU Requests  CPU Limits  Memory Requests  Memory Limits  Age
  ---------                   ----                              ------------  ----------  ---------------  -------------  ---
  kube-system                 canal-gjxwj                       25m (2%)      0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 coredns-86b698fbb6-8q542          50m (5%)      0 (0%)      50Mi (2%)        170Mi (9%)     29d
  kube-system                 coredns-86b698fbb6-hqpmj          50m (5%)      0 (0%)      50Mi (2%)        170Mi (9%)     29d
  kube-system                 kube-proxy-lhxdd                  0 (0%)        0 (0%)      0 (0%)           0 (0%)         29d
  kube-system                 metrics-server-bcffdb84f-mjt64    100m (10%)    0 (0%)      200Mi (10%)      0 (0%)         2m36s
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests     Limits
  --------           --------     ------
  cpu                225m (22%)   0 (0%)
  memory             300Mi (15%)  340Mi (18%)
  ephemeral-storage  0 (0%)       0 (0%)
  hugepages-2Mi      0 (0%)       0 (0%)
```

Enfin le nouveau serveur Metrics intégré a été développé et déployé davantage. Le projet **Prometheus** par exemple est couramment utilisé pour collecter des métriques, en combinaison avec **Grafana** pour afficher graphiquement les métriques en question ont pu faire un succès auprès des entreprises qui adopte Kubernetes.

Chapitre 10

Sécurité

10 Laboratoires

Exercice 10.1 : Authentification et autorisation

Les clusters Kubernetes ont deux types de comptes de service d'utilisateurs : comptes et utilisateurs normaux, mais les utilisateurs normaux sont supposés être gérés par un service externe. Il n'y a aucun objet pour les représenter et ils ne peuvent pas être ajoutés via un appel API, mais des comptes de service peuvent être ajoutés.
Nous utiliserons **RBAC** pour configurer l'accès aux actions dans un espace de noms pour un nouvel utilisateur, le développeur grassa, qui travaillera sur un nouveau projet.

1. Créons deux espaces de noms, un pour la production et l'autre pour le développement.

```
controlplane $ kubectl create ns development
```

```
namespace/development created
```

```
controlplane $ kubectl create ns production
```

```
namespace/production created
```

2. Affichons le cluster courant et le contexte disponible. Le contexte nous permet de configurer le cluster à utiliser, l'espace de noms et l'utilisateur pour les commandes comme **kubectl** de manière simple et cohérente.

```
controlplane $ kubectl config get-contexts
```

```
CURRENT   NAME                          CLUSTER     AUTHINFO        NAMESPACE
*         kubernetes-admin@kubernetes   kubernetes  kubernetes-admin
```

3. Je vais créer un nouvel utilisateur à mon nom grassa et lui attribuez un mot de passe

```
controlplane $ sudo useradd -s /bin/bash grassa
controlplane $ sudo passwd grassa
```

```
New password:
Retype new password:
passwd: password updated successfully
```

4. Je vais générer une clé privée puis une demande de signature de certificat (CSR) pour grassa.

```
controlplane $  openssl genrsa -out grassa.key 2048
```

```
Generating RSA private key, 2048 bit long modulus (2 primes)
.............+++++
....................+++++
e is 65537 (0x010001)
```

```
controlplane $ openssl req -new -key grassa.key -out grassa.csr  \
 -subj  "/CN=grassa/O=development"
controlplane $ ls
```

```
cluster-api-config  filesystem  grassa.csr  grassa.key  snap
```

5. À l'aide de la requête nouvellement créée, générons un certificat auto-signé à l'aide du protocole x509. J'utiliserai les clés CA pour le cluster Kubernetes et définir une expiration de 45 jours.

```
controlplane $ sudo openssl x509 -req -in grassa.csr \
> -CA  /etc/kubernetes/pki/ca.crt \
> -CAkey  /etc/kubernetes/pki/ca.key \
> -CAcreateserial \
> -out grassa.crt -days 45
```

```
Signature ok
subject=CN = grassa, O = development
Getting CA Private Key
```

6. Mettons à jour le fichier de configuration d'accès pour référencer la nouvelle clé et le nouveau certificat. Normalement, je devrais les déplacer vers un répertoire sûr au lieu de les laisser dans le répertoire root.

```
controlplane $  kubectl config set-credentials grassa \
> --client-certificate=/root/grassa.crt \
> --client-key=/root/grassa.key
```

```
User "grassa" set.
```

7. Affichons la mise à jour de notre fichier d'informations d'identification. Utilisons pour cela **diff** pour comparer avec la copie que nous avons faite plus haut : « **cp $HOME/.kube/config $HOME/cluster-api-config** »

controlplane $ diff cluster-api-config .kube/config

```
15a16,19
> - name: grassa
>   user:
>     client-certificate: /root/grassa.crt
>     client-key: /root/grassa.key
```

8. Nous allons maintenant créer un contexte nommé « **grassa-context** ». Pour cela, nous aurons besoin du nom du cluster, de l'espace de noms et du nom commun (CN) de l'utilisateur que nous avons déjà défini lors des étapes précédentes.

controlplane $ kubectl config set-context **grassa-context** \
> --cluster=kubernetes \
> --namespace=development \
> --user=grassa

```
Context "grassa-context" created.
```

9. Essayons voir si on pouvait afficher les pods dans **grassa-context** . Nous obtiendrons une erreur qui indique qu'on n'a pas les autorisations nécessaire pour afficher les pods dans ce contexte. Pour y remédier on va utiliser **RBAC**. RBAC (Role-Based Access Control) est un mécanisme utilisé dans Kubernetes pour contrôler l'accès aux ressources et aux opérations dans un cluster Kubernetes. Son rôle est de définir et de gérer les autorisations des utilisateurs et des services dans un environnement Kubernetes en fonction de leurs rôles. C'est ce qu'on va essayer de faire par la suite.

controlplane $ kubectl --context=grassa-context get pods

```
Error from server (Forbidden): pods is forbidden: User "grassa" cannot list resource "pods" in API
group "" in the namespace "development"
```

10. Vérifions d'abord que le contexte a été correctement défini.

controlplane $ kubectl config get-contexts

```
CURRENT   NAME                          CLUSTER      AUTHINFO          NAMESPACE
          grassa-context                kubernetes   grassa            development
*         kubernetes-admin@kubernetes   kubernetes   kubernetes-admin
```

11. Vérifions à nouveau les modifications récentes apportées au fichier de configuration d'accès au cluster.

controlplane $ diff cluster-api-config .kube/config

```
9a10,14
>    namespace: development
>    user: grassa
>   name: grassa-context
> - context:
>    cluster: kubernetes
15a21,24
> - name: grassa
>   user:
>    client-certificate: /root/grassa.crt
>    client-key: /root/grassa.key
```

12. Nous allons maintenant créer un fichier YAML pour associer les droits RBAC à un espace de noms et un rôle particuliers. Puis créons l'objet **Role**

controlplane $ vi role-dev.yaml

role-dev.yaml

```yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: development
  name: developer
rules:
- apiGroups: ["", "extensions", "apps"]
  resources: ["deployments", "replicasets", "pods"]
  verbs: ["list", "get", "watch", "create", "update", "patch", "delete"]
# On peut utiliser ["*"] pour tous les verbs
```

controlplane $ kubectl create -f role-dev.yaml

```
role.rbac.authorization.k8s.io/developer created
```

13. Nous créons maintenant un **RoleBinding** pour associer le **rôle** que nous venons de créer à un utilisateur. Créons l'objet RoleBinding une fois le fichier créé

controlplane $ vi rolebind.yaml

rolebind.yaml

```yaml
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: developer-role-binding
  namespace: development
subjects:
- kind: User
  name: grassa
  apiGroup: ""
roleRef:
  kind: Role
  name: developer
  apiGroup: ""
```

controlplane $ kubectl create -f rolebind.yaml

```
rolebinding.rbac.authorization.k8s.io/developer-role-binding created
```

14. Testons à nouveau le contexte grassa. Cette fois, ça marche. Aucun pod n'est en cours d'exécution, nous n'avons trouvé aucune ressource.

controlplane $ kubectl --context=grassa-context get pods

```
No resources found in development namespace.
```

15. Créons un nouveau pod nginx , vérifions qu'il est bien crée, puis supprimons-le.

controlplane $ kubectl --context=grassa-context create deployment nginx --image=nginx

```
deployment.apps/nginx created
```

controlplane $ kubectl --context=grassa-context get pods

```
NAME                      READY   STATUS    RESTARTS   AGE
nginx-7854ff8877-nzbbn    1/1     Running   0          14s
```

controlplane $ kubectl --context=grassa-context delete deployment nginx

```
deployment.apps "nginx" deleted
```

16. Nous allons maintenant créer un contexte différent pour les systèmes de production que j'ai nommé « **Prodgrass-context** » . Le rôle aura uniquement la capacité d'afficher, mais pas de créer ou de supprimer des ressources. Commencez par copier et modifier les fichiers YAML Role et RoleBindings.

controlplane $ cp role-dev.yaml role-prod.yaml
controlplane $ vim role-prod.yaml

role-prod.yaml

```yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: production
  name: dev-prod
rules:
- apiGroups: ["", "extensions", "apps"]
  resources: ["deployments", "replicasets", "pods"]
  verbs: ["get", "list", "watch" ]
```

controlplane $ cp rolebind.yaml rolebindprod.yaml
controlplane $ vim rolebindprod.yaml

rolebindprod.yaml

```yaml
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: production-role-binding  #<-- modifier par production
  namespace: production          #<-- même chose ici
subjects:
- kind: User
  name: grassa
  apiGroup: ""
roleRef:
  kind: Role
  name: dev-prod                 #<-- changement ici par dev-prod
  apiGroup: ""
```

17. Créons les deux nouveaux objets.

controlplane $ kubectl create -f role-prod.yaml

```
role.rbac.authorization.k8s.io/dev-prod created
```

controlplane $ kubectl create -f rolebindprod.yaml

```
rolebinding.rbac.authorization.k8s.io/production-role-binding created
```

18. Créons le nouveau contexte pour une utilisation en production.

controlplane $ kubectl config set-context **Prodgrass-context** \
> --cluster=kubernetes \
> --namespace=production \
> --user=grassa

```
Context "Prodgrass-context" created.
```

19. Si l'on essaie de créer un Pod en production. On nous rejette, l'utilisateur grassa ne peut pas créer des pods avec ce nouveau contexte mais peut par contre les affiché s'il en existe.

controlplane $ kubectl --context=Prodgrass-context create deployment nginx --image=nginx

```
error: failed to create deployment: deployments.apps is forbidden: User "grassa" cannot create
resource "deployments" in API group "apps" in the namespace "production"
```

20. Affichons les détails du rôle « dev-role ». Dans la partie **Verbs** on peut voir les droits dans l'espace de nom production et **create** (créer) ne fait pas partie des droits autorisés.

controlplane $ kubectl -n production describe role dev-prod

```
Name:         dev-prod
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources                Non-Resource URLs  Resource Names  Verbs
  ---------                -----------------  --------------  -----
  deployments              []                 []              [get list watch]
  pods                     []                 []              [get list watch]
  replicasets              []                 []              [get list watch]
  deployments.apps         []                 []              [get list watch]
  pods.apps                []                 []              [get list watch]
  replicasets.apps         []                 []              [get list watch]
  deployments.extensions   []                 []              [get list watch]
  pods.extensions          []                 []              [get list watch]
  replicasets.extensions   []                 []              [get list watch]
controlplane $
```

controlplane $ kubectl config get-contexts

```
CURRENT   NAME                          CLUSTER        AUTHINFO          NAMESPACE
          Prodgrass-context             kubernetes     grassa            production
          grassa-context                kubernetes     grassa            development
*         kubernetes-admin@kubernetes   kubernetes     kubernetes-admin
```

Enfin si on peut se permettre de faire un petit résumé sur les ressources **RBAC** on peut dire que :

- Un **ClusterRole|Role** définit un ensemble d'autorisations et l'endroit où ils sont disponibles, dans l'ensemble du cluster ou dans un seul espace de noms.

- Un **ClusterRoleBinding|RoleBinding** connecte un ensemble d'autorisations à un compte et définit où il est appliqué, dans l'ensemble du cluster ou dans un seul espace de noms.

Pour cette raison, il existe 4 combinaisons RBAC différentes mais seulement 3 valides :

Rôle + RoleBinding (disponible dans un seul espace de noms, appliqué dans un seul espace de noms)
ClusterRole + ClusterRoleBinding (disponible à l'échelle du cluster, appliqué à l'échelle du cluster)
ClusterRole + RoleBinding (disponible à l'échelle du cluster, appliqué dans un seul espace de noms)
Role + ClusterRoleBinding (PAS POSSIBLE : disponible dans un seul espace de noms, appliqué à l'échelle du cluster) donc non faisable.

Exercice 10.2 : Travailler avec TLS

Travailler avec TLS (Transport Layer Security) dans Kubernetes implique de sécuriser la communication entre les différents composants du cluster Kubernetes et les services externes. Voici un exemple concret dont on utilise TLS dans Kubernetes :

1. Tout d'abord, On va se basé sur les certificats TLS générer avec openSSL lors de la section précédente :

controlplane $ ls /root/

```
filesystem  grassa.crt  grassa.csr  grassa.key  snap
```

2. Ensuite Stockons les certificats TLS générés en toute sécurité. Puis on va utiliser Les secrets Kubernetes sont couramment utilisés pour stocker des informations sensibles telles que des certificats. C'est pour cela on va créer un objet Kubernetes **Secret** ayant comme nom **my-tls-secret** contenant les certificats TLS.

controlplane$ kubectl create secret tls my-tls-secret --cert=/root/grassa.crt \
--key=/root/grassa.key

```
secret/my-tls-secret created
```

controlplane $ k describe secrets my-tls-secret

```
Name:       my-tls-secret
Namespace:   default
Labels:      <none>
Annotations:  <none>
Type:  kubernetes.io/tls
Data
====
tls.crt:  1017 bytes
tls.key:  1675 bytes
```

3. Configurons à présent TLS pour **Ingress**. Ingress est une ressource API qui gère les règles de routage du trafic HTTP et HTTPS depuis l'extérieur du cluster vers les services à l'intérieur du cluster. Donc on doit utiliser Ingress pour exposer nos services, et configurer aussi la terminaison TLS au niveau du **contrôleur Ingress**. Un contrôleur Ingress est un composant dans un cluster Kubernetes qui surveille les objets Ingress et met en œuvre les règles de routage spécifiques dans ces objets.
   Il existe différents contrôleurs Ingress disponibles pour Kubernetes, mais le plus populaire étant **NGINX Ingress Controller**
   Voici un exemple de configuration de la terminaison TLS dans une ressource Ingress :

controlplane $ vim my-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  tls:
  - hosts:
    - example.com
    secretName: my-tls-secret
  rules:
  - host: example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-service
            port:
              number: 80
```

controlplane $ kubectl apply -f my-ingress.yaml

```
ingress.networking.k8s.io/my-ingress created
```

4. Voici un exemple de création d'un pod nginx dans Kubernetes utilisant le secret TLS **my-tls-secret**. Notons que **imagePullSecrets** est utilisé pour spécifier les secrets d'authentification nécessaires pour que le conteneur puisse utiliser l'image.

controlplane $ vim my-pod.yaml
controlplane $ kubectl apply -f my-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx
      ports:
        - containerPort: 443
  volumes:
    - name: tls-certs
      secret:
        secretName: my-tls-secret
  imagePullSecrets:
    - name: my-image-pull-secret
  restartPolicy: Always
```

controlplane $ kubectl get po

```
NAME    READY   STATUS    RESTARTS   AGE
my-pod  1/1     Running   0          8s
```

5. **Important à savoir sur TLS:**

   Configuration de TLS pour les services : si nos services communiquent entre eux en **interne** au sein du cluster, nous pouvons configurer TLS pour la communication de service à service. Cela implique d'utiliser par exemple un service comme **my-service.** C'est pourquoi j'ai laisser le port **80**. Regardez la description ci-dessous

   Communication sécurisée avec des services externes : si nos services communiquent avec des services externes via TLS, il faut s'assurer qu'ils sont configurés pour utiliser

les certificats TLS appropriés pour une communication sécurisée. On utilise https dans cas.

controlplane $ kubectl describe ingress my-ingress

```
Name:          my-ingress
Labels:        <none>
Namespace:     default
Address:
Ingress Class:   <none>
Default backend:  <default>
TLS:
  my-tls-secret terminates example.com
Rules:
  Host        Path  Backends
  ----        ----  --------
  example.com
              /   my-service:80 (<none>)
Annotations:   <none>
Events:        <none>
```

controlplane $ kubectl get ingress

```
NAME         CLASS    HOSTS        ADDRESS   PORTS    AGE
my-ingress   <none>   example.com          80, 443   8m46s
```

6. Pour tester l'accès à notre service Ingress. Utilisons l'outil en ligne de commande **curl**. Mais d'abord voyons voir si l'on peut résoudre le nom de domaine **example.com** en local avec l'outil **nslookup.** Bien entendu si l'on veut qu'un nom de domaine comme « example.com » soit résolvable depuis l'extérieur, il faut l'acheter chez un fournisseur comme OVH. Le nom de domaine ne doit pas être réservé évidemment.

controlplane $ nslookup example.com

```
Server:        8.8.8.8
Address:       8.8.8.8#53

Non-authoritative answer:
Name:   example.com
Address: 93.184.216.34
Name:   example.com
Address: 2606:2800:220:1:248:1893:25c8:1946
```

controlplane $ curl https://example.com

```
<!doctype html>
<html>
<head>
   <title>Example Domain</title>
```

ANNEXE

Exercice A.1 : CKA Simulator Kubernetes 1.29

Avant de passer l'examen

Utilisez cet exercice (**voir lien en bas de la page**) comme ressource après avoir terminé **le cours**, mais avant de passer l'examen. Consultez les ressources de la page officielle de Kubernetes. Entraînez-vous à créer des fichier YAML et à travailler en ligne de commande pour faciliter la révision et la préparation de l'examen.

1. À l'aide d'un navigateur, accédez à https://www.cncf.io/certification/cka/ et lisez la description du programme.
2. Dans la section Ressources d'examen, ouvrez l'aperçu du programme et le manuel du candidat dans de nouveaux onglets. Ces deux doit être lu et compris avant de passer l'examen.
3. Accédez à l'onglet Aperçu du programme. Vous devriez voir des liens vers des informations sur le domaine pour les différentes versions de l'examen.

Sélectionnez la dernière version, dans notre cas **CKA V1.29**.

4. Essayer d'appliquer les exercices dans le cours ci-joint. N'hésitez pas d'aller à chaque fois sur le site officiel de Kubernetes, vous trouverez plus d'informations, il est à ma fois exhaustive et à jour.
5. Sachez qu' une fois que vous êtes dans l'examen vous n'avez droit qu'aux URLs autorisés par l'examen et je parle ici uniquement le site officiel de Kubernetes, donc recherchez des exemples YAML pour chaque domaine ou élément de compétence. Assurez-vous que vous êtes dans la bonne la version de l'examen que vous passez, car le YAML peut changer d'une version à l'autre. IL faut se familiariser avec le site officiel de Kubernetes pour retrouver chaque bon exemple, afin que vous puissiez retrouver la page pendant l'examen le plus rapidement possible.
6. À l'aide d'un minuteur, voyez combien de temps il vous faut pour créer et vérifier les objets répertoriés dans le site donné ci-dessous . Notez l'heure. Essaye encore et voyez à quelle vitesse vous pouvez terminer et tester chaque étape.
7. Malheureusement l'examen CKA est disponible uniquement en anglais.

Voici le site pour s'entrainer (vous trouverez des questions ainsi que leurs réponses)

http://n.grassa.free.fr/cka/Killer%20Shell%20-%20Exam%20Simulators.html

Enfin Je vous souhaite bon courage.

Webographie

[1] https://kubernetes.io/docs/home/
[2] https://kubernetes.io/docs/concepts/services-networking/ingress/
[3] https://killercoda.com/login
[4] https://dashboard.ngrok.com/get-started/setup/windows
[5] https://nigelpoulton.com/explained-kubernetes-service-ports/
[6] https://sso.linuxfoundation.org/
[7] https://kodekloud.com/learning-path/cka/
[8] https://www.udemy.com/course/certified-kubernetes-administrator-with-practice-tests/